Predicting Student Dropout, with a Fairness Audit

Capstone project, Postgraduate Diploma in AI and Machine Learning, Pillar 5.

Jeryl Donato Estopace

Predicting whether a student will leave higher education, using only what the school knows on the day they enrol, and then auditing the model for bias before anyone trusts it.

All code, notebooks with saved outputs, data dictionary, and saved models are in the public repository.

https://github.com/jeryldev/pgdaiml-capstone

What is in this file

The project is organised around four mistakes made, found, and fixed, each of which changed a conclusion. They are written up in the report and the notebooks rather than quietly corrected.

Dataset: Realinho, V., Vieira Martins, M., Machado, J., and Baptista, L. (2021). *Predict Students' Dropout and Academic Success*. UCI Machine Learning Repository. DOI 10.24432/C5MC89. Licensed CC BY 4.0.

Predicting Student Dropout, with a Fairness Audit

Capstone report, PGD AI/ML, Pillar 5. This is a machine learning project from start to finish. It predicts which students are at risk of dropping out of higher education. It uses only what the school knows when a student first enrolls. Then it checks the model for bias before anyone trusts it.

Repository: **https://github.com/jeryldev/pgdaiml-capstone**

The full code lives there. There is one notebook per step, and the outputs are saved. So you can check every number in this report without running anything. This report tells the whole story, from how I framed the problem to how I tested for fairness. It also covers the parts I got wrong on the first pass.

Executive Summary

Every year some students leave higher education before finishing their degree. The cost falls on three sides. It hurts the student, the school, and society. A tool that spots who is at risk early, around the time a student enrolls, lets a support team reach those students before they leave.

This project builds that tool on real data from a Portuguese polytechnic. Then it does the harder work of checking whether the tool is fair.

The final model is a logistic regression. This is a simple method that adds up weighted signals to score each student. I set a cutoff on purpose to match how many students the school can actually support. At that cutoff the model catches **79.6 percent** of the students who go on to drop out, using only what is known at enrollment, but it flags women less reliably than men. The version I recommend shipping adds the Step 5 fairness fix. It brings recall to **71.8 percent** and closes the gender recall gap from 0.177, which a bootstrap calls real, to 0.009, which the same bootstrap cannot tell apart from zero. That costs about 8,000 EUR per 1,000 students. It is simple enough to explain its own decisions. In this project that turned out to matter more than any accuracy figure.

Four findings are worth leading with. Every one of them came from fixing a mistake.

The model is not mostly a money model. To find out what drives the score I used SHAP. SHAP measures how much each feature pushed a given prediction up or down. At first that weight was spread across many tiny yes or no columns. Once I added it back up onto the real feature each column came from, the picture changed. The strongest driver is **which course a student enrolled in**, far ahead of everything else. Whether tuition is paid sits second. **What the student's mother does for a living** sits level with holding a scholarship, right behind it. So the model is mostly spotting students who started from further back, not students in trouble.

The fairness verdict is not "everything fails." My first check looked at four groups we cared about. These were gender, age, scholarship, and debt. The check asked whether the model flags each group at the same rate. By that test all four failed. None stayed inside the usual safe ratio of 0.8. But that was the wrong question for three of the four. Students who owe the school money drop out 76 percent of the time, against 34 percent for the rest. A model that flagged them at the same rate as everyone else would be ignoring a real 42-point gap on purpose. The fix is to put the model's flagging gap next to the real gap in the world. Only debt flags below its real gap. The model widens the gaps for scholarship, gender, and age. Gender is the one that matters, because gender is a protected trait. Scholarship is not, and its flagging tracks real risk. The age gap may involve a protected trait and is not yet resolved.

The real harm is a recall gap. Recall means how many of the true dropouts the model actually flags. The model catches **87.7 percent of male dropouts and 70.0 percent of female ones**. Three in ten women who go on to leave are never flagged at all. You cannot see this by looking at flagging rates. It shows up only when you stop counting flags and start counting mistakes.

And rigor does not carry over on its own. In Step 4 I refused to call a 0.005 lead real, because the score swung by 0.020 from one slice of data to the next. Then in Step 5 I quoted fairness numbers to three decimal places off just 130 female dropouts. To check those numbers I used a bootstrap. A bootstrap re-draws the test data thousands of times to see how much a number wobbles by chance. It showed the two fixes are **tied on fairness**. You cannot tell them apart. The only real difference is what each one costs, eight points of recall against nineteen. The harm survived the check. My reason for picking one fix over the other did not.

The honest conclusion is that the model works and should not be trusted blindly. It belongs in the hands of a support team. It should be a ranked list that starts a conversation, not a verdict stamped on a student's record.

Step 1. Problem Framing

The problem, and who it helps

The data comes from the Polytechnic Institute of Portalegre (IPP), a public higher education school in Portugal. The school and its research team built this dataset to reduce dropout and failure. The plan was to spot at-risk students early so support can be put in place. This project follows the same goal.

The model serves the tutoring and support staff. They cannot watch every student closely, so they need to spend their limited time on those most at risk. The model turns a long list of students into a short ranked one. It puts the students worth contacting first at the top. A correct flag reaches a student who would otherwise slip away. A wrong flag wastes staff time, or misses a student who needed help.

The task, and why this kind

The original data has three outcomes. These are Dropout, Graduate, and Enrolled. Enrolled means the student was still studying when the record was taken. Their final outcome is not settled yet. A student with no settled outcome cannot teach the model what dropout looks like. So those 794 rows are removed. That leaves two settled outcomes, which fits a yes or no question. The task is **binary classification**, Dropout against Graduate, on 3,630 students at a 39.1 percent dropout rate. Binary classification just means sorting each student into one of two boxes.

The answer we want is a category. A student either drops out or does not. Regression is out because it predicts a number. Clustering is out too because it finds patterns in data with no labels. Here the labels are known. So this is a classification problem.

How success is measured

The model gives back a score, not a plain yes or no. To turn that score into a decision you need a cutoff. A cutoff is the line above which a student gets flagged. **The cutoff is a choice, not a given.** That idea drives everything later, so it is worth saying which measures depend on it and which do not.

Two measures choose the model. Neither one depends on where the cutoff sits.

- **PR-AUC** is built from precision and recall. Precision is how often a flag is correct. Recall is how many of the true dropouts get flagged. PR-AUC ignores the graduates the model correctly leaves alone. So it stays honest on an uneven 39 to 61 split. This is the score the tuning tries to raise.
- **ROC-AUC** is the chance that the model gives a random dropout a higher score than a random graduate. In plain terms it measures how well the model ranks students. That matches how the tool is actually used.

One measure judges the live system, once a cutoff exists.

- **Recall on dropout.** Of the students who truly drop out, how many did the model flag? A missed student is the costly mistake. This is the number the school cares about, and the number I would report to them.

But recall cannot pick a model, and the reason matters. **Recall does not belong to a model alone. It belongs to a model plus a cutoff.** You can push recall to 1.0 on any model just by flagging every student. So PR-AUC and ROC-AUC pick the model. Step 4 picks the cutoff on purpose. Only then does recall mean anything.

Accuracy is reported but not trusted. With a 39 percent dropout rate, a lazy model that calls everyone a graduate is right 61 percent of the time while catching no one.

Business value

Value is the students the school keeps, minus the cost of reaching out to flagged students. Three stated assumptions keep the estimate honest.

- A kept student is worth about **2,100 euros**. This comes from the Portuguese public tuition cap of roughly 697 euros a year over about three years. It is a low estimate on purpose. Tuition is a number I can cite. The wider loss to the student and to society is larger but harder to defend.
- A flag starts a conversation. It does not keep the student on its own. So 2,100 euros is the most a caught student can be worth. I scale it down by how often outreach actually works, taken here as 3 in 10.
- Outreach costs roughly **50, 150, or 300 euros** per flagged student. I give a range because the real figure depends on the school.

These numbers are not decoration. Step 4 uses them directly to choose the cutoff. That is the one place in this project where cost and benefit actually meet.

Scope

The model fits an IPP style Portuguese polytechnic. It covers 17 undergraduate degrees, from 2008-09 through 2018-19. It is not a general dropout predictor for any school in any country. The method carries over even though the model does not.

Step 2. Data Understanding

Source and citation

The dataset is the UCI set *Predict Students' Dropout and Academic Success* (Realinho, Vieira Martins, Machado, and Baptista, 2021). It was donated in December 2021 under a CC BY 4.0 license, DOI 10.24432/C5MC89. The Portuguese program SATDAP funded it, grant POCI-05-5762-FSE-000191. The creators joined several separate school databases into one table with one row per student. They say they cleaned the data

carefully for odd values, outliers, and missing cells before release. The full citation is in `data/README.md`.

The degrees behind the course codes span a wide mix. They range from Nursing and Veterinary Nursing to Agronomy, Social Service, Journalism, Management, Communication Design, and Basic Education. That spread matters later, because course turns out to be the strongest predictor in the model.

Overview

4,424 students, 37 columns. There are no missing cells and no duplicate rows. That is the providers' careful cleaning, not luck, and I checked it rather than assumed it. The outcome splits into Graduate 2,209, Dropout 1,421, and Enrolled 794.

A full column-by-column data dictionary is saved at **`data/data_dictionary.md`**. It lists each column's type, range, unit, and missing count, and it is built straight from the data in notebook 02.

Feature types

Type	Count	Meaning
True numbers	6	Real quantities where order and distance matter, like admission grade and age
Yes or no flags	8	Stored as 1 or 0, like gender, scholarship holder, and debtor
Coded categories	9	Numbers that stand for categories, like course and parental qualification. The number is a label, not an amount
Ordered count	1	Application order, a rank from **0 = first choice** to 9 = last
Curricular, leakage	12	First and second semester performance, kept out of the model
Target	1	Dropout, Graduate, Enrolled

The coded categories are the trap. A course coded 9500 is not bigger than one coded 33. It is just a different course. So it is wrong to treat these as real numbers, to scale them, or to measure plain correlation on them. They are handled as categories throughout.

First look at disparity

Four groups drive the fairness work in Step 5. These are gender, scholarship, debt, and age. Against an overall dropout rate of 39.1 percent, the gaps between groups are already large before any model exists.

Group	Dropout rate	n
Men	56.1 percent	1,249
Women	30.2 percent	2,381
No scholarship	48.4 percent	2,661
Scholarship holder	13.8 percent	969
Debtor	75.5 percent	413
Not a debtor	34.5 percent	3,217

Group	Dropout rate	n
Age 17 to 20	26.1 percent	2,080
Age 21 to 23	40.6 percent	473
Age 24 to 30	66.5 percent	499
Age 31 plus	61.4 percent	578

These numbers come back in Step 5. They are the reason I had to redo the fairness audit. A gap this large already exists in the world. A model does not invent it. Whether the model makes the gap *worse* is a separate question. It needs a different measure to answer.

One column stands apart. Students **behind on tuition drop out 94.0 percent of the time**, against 30.7 percent for those who are paid up. Only 486 students are behind. That is not a normal background fact. A student who has stopped paying is often a student already on the way out. It is almost the outcome itself. I keep it but watch it closely, and Step 3 measures exactly what the model would lose without it.

Step 3. Preprocessing, EDA, and Feature Engineering

Cleaning and the leakage guard

First the plain data checks. Step 2 found no missing cells and no duplicate rows, and I confirmed both in code rather than trusting them. A range check on the six numerics turned up nothing impossible, only a long tail of mature-student ages that reaches to 70. Those ages are real records, not errors, and age is the strongest numeric predictor, so no rows are cut. Inside the pipeline the numerics are scaled with StandardScaler and the coded categories are one-hot encoded, both fit on the training data only.

The twelve curricular columns record how many courses a student took, passed, and failed in the first and second semesters. That record predicts dropout almost perfectly, because a student failing courses is a student already leaving. A model built on it would score high and help no one. So all twelve are dropped. The model then sees only what is known at enrollment. This trades raw accuracy for a warning that arrives in time to act on. That trade is on purpose.

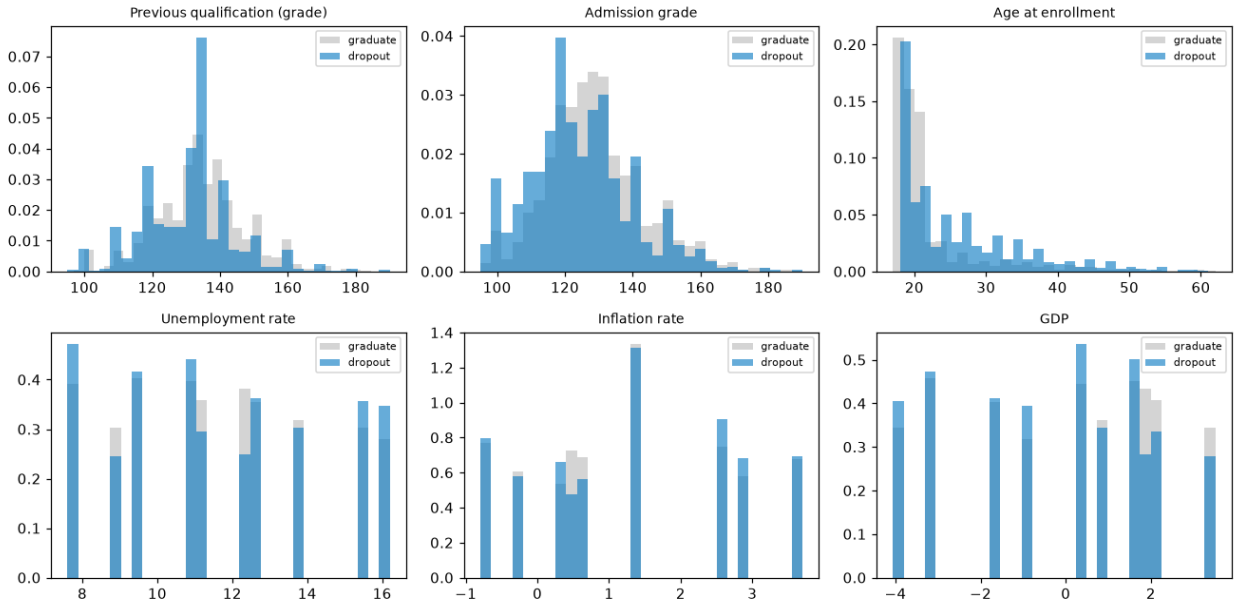
The split comes first

I split the data into a training set and a test set before any feature work that looks at the outcome. The split keeps the same dropout rate in both parts and uses a fixed seed so it repeats. That leaves 2,904 training students and 726 test students.

Every decision in this notebook is made on the training data, including the tuition check below. My first version of that check compared models on the held-out test set. That was a quiet leak. Any decision made by looking at test results feeds test information back into the model, no matter how small the decision seems.

What predicts dropout

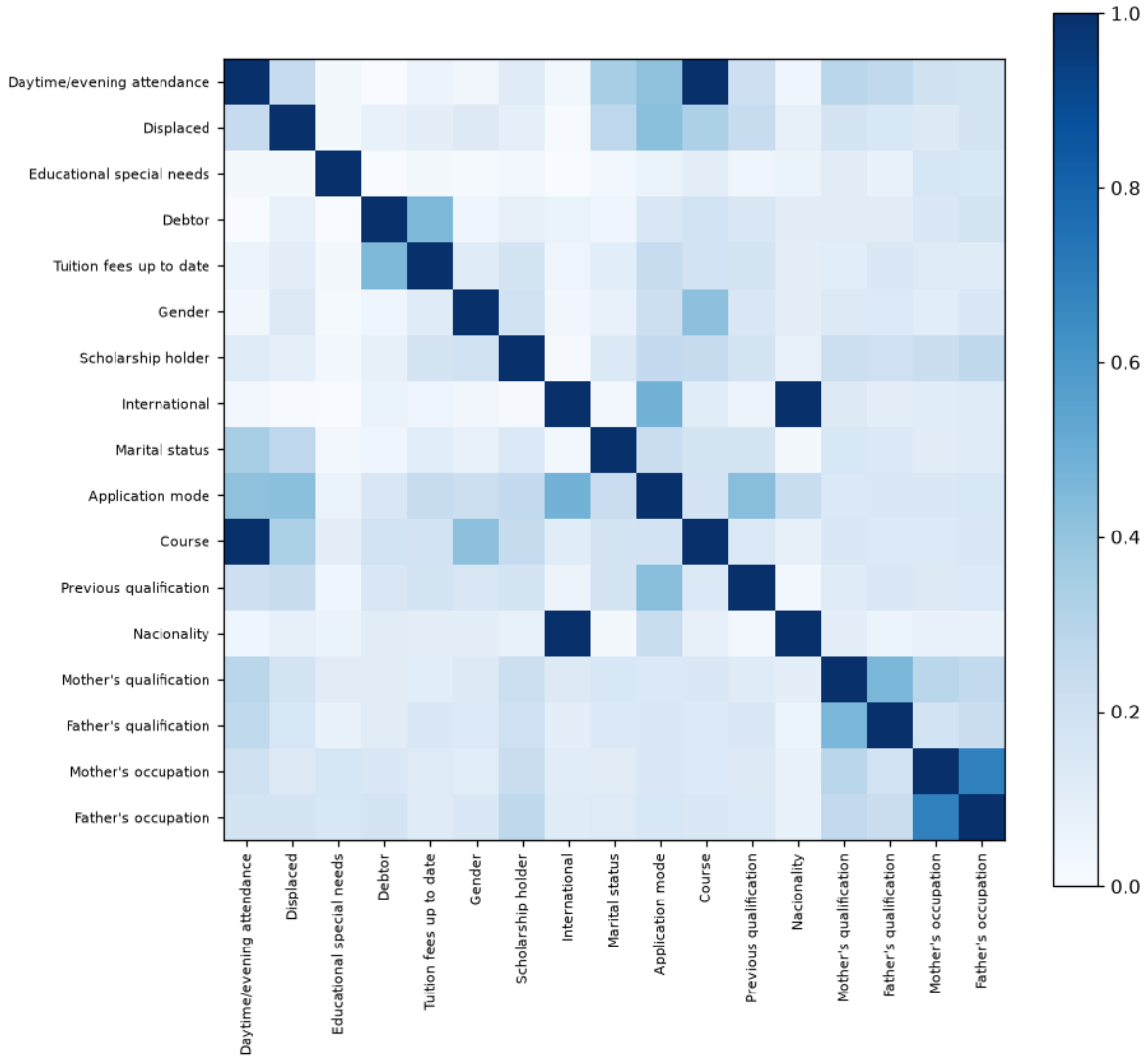
True numbers. Grades and age are lopsided, not bell-shaped, so I used the Mann-Whitney U test. This test compares two groups without assuming any particular shape. Age has the strongest link. Older students drop out more. Admission grade and previous qualification grade come next. Unemployment and inflation show no real difference ($p = 0.75$ and 0.21).



Categories. For these I used two tools together. The chi-square test asks whether a link is real. Cramér's V then rates that link on a scale from 0 to 1, where higher means stronger. Used together they rank categories by a link that is both real and strong. On 2,904 training rows almost every link is real, so Cramér's V does the real ranking.

Feature	Cramér's V
Tuition fees up to date	0.437
Course	0.346
Application mode	0.330
Scholarship holder	0.321
Debtor	0.269
Gender	0.255

Overlap between the numbers. I checked whether the number columns repeat each other, using a score called VIF. A high VIF means one column is largely a stand-in for the others. Every VIF on the true numbers is under 2, which is low. The linear model is safe on that front. But VIF only looks at the six number columns. It says nothing about the yes or no columns, or about any feature built later. That blind spot turns out to matter enormously.



The chart above shows two pairs of categories at a perfect **V = 1.000**, meaning each pair moves in lockstep. I noticed that and moved on. That was a mistake.

The tuition status check

Tuition status leads the ranking. Its 94 percent dropout rate makes me suspect it is almost the outcome itself. To test that, I trained a logistic regression with it and without it. I scored both with cross-validation inside the training set. Cross-validation splits the training data into several parts, trains on most of them, and tests on the part left out, then rotates. That way the score never touches the test set.

Version	CV PR AUC	CV ROC AUC
With tuition status	0.817	0.848
Without tuition status	0.765	0.826

Removing it costs five points of PR-AUC. That is too much to give up for a risk that is only a judgement call. It is not a proven leak. The curricular columns were the proven leak. So the tuition flag stays, with a note. A school acting on this model is acting, in large part, on who is behind on payments.

The feature I threw away

This is the part of the project I would defend first. It started as a failure.

I built a **socioeconomic pressure** score. It gave one point each for owing money, falling behind on tuition, and holding no scholarship. The idea felt right and the numbers agreed. It ranked **first** on mutual information, ahead of every raw column in the file. Mutual information measures how much knowing one column tells you about the outcome.

Then the model's weights came back wrong. Not weak. **Wrong.** The model said owing the school money makes a student *less* likely to drop out, and holding a scholarship makes them *more* likely. The data says the opposite, loudly.

The cause was a hidden overlap. The three source flags added up to the score exactly, on every single training row. Take the score, subtract its three flags, and you always get `2`. So the score was not new information. It was three columns the model already had, added together. There was no single right way to split the weight among four columns that move as one. The model just picked one split at random. That random split was what I had been reading as a real weight.

A rank check on the feature table made it visible. A rank check counts how many columns are truly independent versus how many are just combinations of others.

	Old matrix	**Fixed matrix**
Columns	216	**84**
One-hot blocks	9	6
Exact linear dependencies	**19**	**6**

Some overlap is expected. When a category is split into one column per value, the columns in that group always add up to a constant, so each group brings one built-in overlap. Six category groups give six such overlaps. That is normal. The old table had **ten more than it should**. Here is what those ten did.

Feature	**Coefficient, score in matrix**	**Coefficient, score removed**	**Actual dropout rate**
Debtor	**−0.465**	**+1.006**	76% vs 34%
Scholarship holder	**+0.138**	**−1.344**	14% vs 49%
Tuition fees up to date	−1.480	−2.913	31% vs 94%

Two of the signs flip. And the reason the feature had to go is not the one I expected. It barely moved accuracy at all, because it never held information the model did not already have. What it did was make the model **unreadable**. Every explanation in Step 5 is read straight off these weights. So a flipped sign here becomes a sentence that tells a student the opposite of the truth.

A feature that costs nothing and explains nothing is not a harmless extra. It is a liability.

The same rank check then caught two more repeats. The Cramér's V chart had already flagged both at 1.000.

- `International` is the same as saying the nationality is not Portuguese. It matches on **100 percent** of rows. It is the same column twice.
- `Daytime/evening attendance` is fixed entirely by `Course`. **Zero of 17** courses run both a day class and an evening class.

Features that replace, instead of repeat

The rule that came out of this is short. **If a new feature is built from columns that stay in the model, it adds nothing to a linear model.** The model already holds that information. A new feature earns its place only if it replaces its source columns, or if it says something the raw columns cannot say on their own.

Feature	What it does	Replaces
`mother education tier`, `father education tier`	Sorts about 30 qualification codes onto a 0-to-3 ladder, keeping the order the codes throw away	Both parental qualification columns (63 dummies → 2)
`first generation`	On when neither parent reached higher education	Nothing. This is genuinely new. You cannot see it until you read the two columns together
`mother isco`, `father isco`	Groups the many occupation codes into a handful of broad job families	Both parental occupation columns (78 dummies → 24)
`application route`	Groups 18 application codes into 5 routes an admissions officer would recognise	Application mode
`mature entry`	On when age is 23 or over, Portugal's *Maiores de 23* route. Age enters as a straight line, but risk jumps at that route, and a straight line cannot bend	Nothing. It lets the age term bend

All of this lives in `src/features.py`. So Steps 4 and 5 read one definition instead of passing copies around. None of it looks at the outcome.

Feature selection, actually applied

Here is the mutual information ranking on the features the model actually uses. Again, this measures how much each feature tells you about the outcome.

Feature	MI
Tuition fees up to date	0.1033
Age at enrollment	0.0663
Course	0.0623
Scholarship holder	0.0575
Previous qualification (grade)	0.0559

Feature	MI
mature entry	**0.0534**

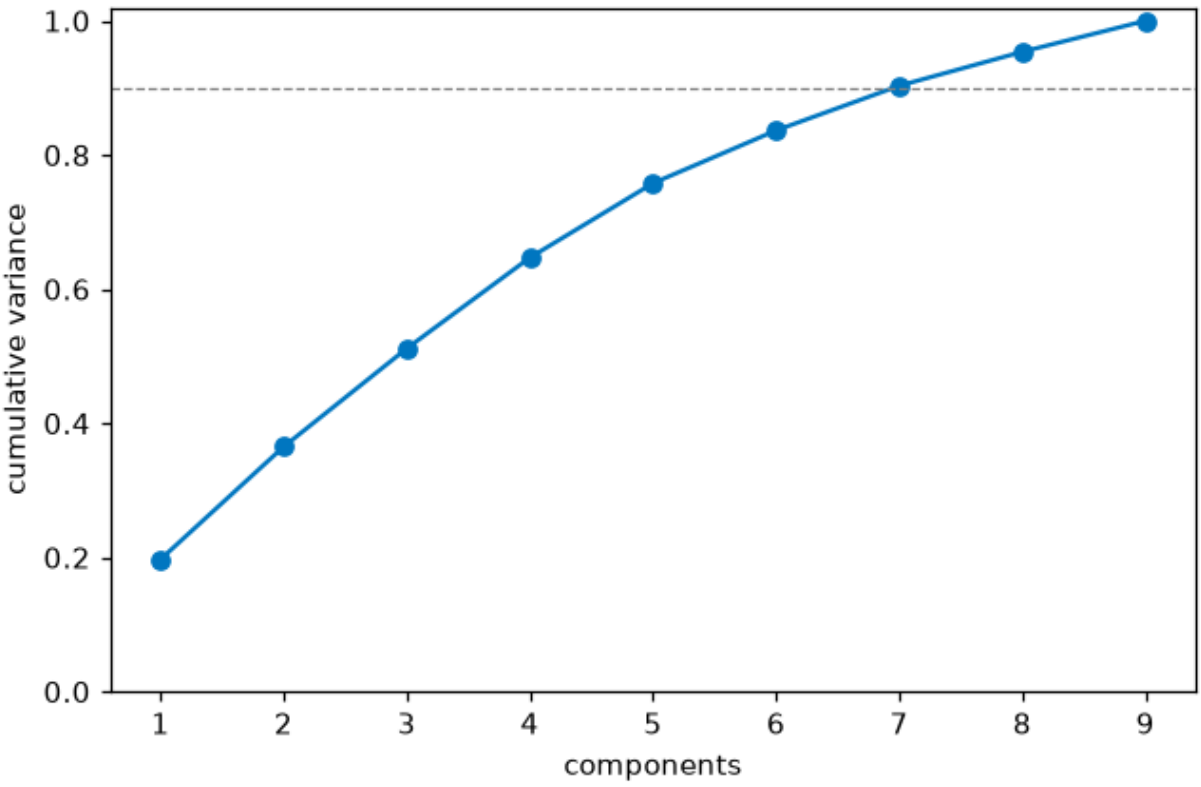
`mature entry` lands sixth, below the raw age column it bends. That is the honest order. Mutual information already sees the whole age signal on its own. Whether the bend earns its place is a modeling question. This ranking does not settle it.

Last time I printed a ranking like this, wrote a sentence about which features were weakest, and then trained on every single one of them anyway. **A ranking that changes nothing is not feature selection. It is just a chart.** So this time four columns actually come out. They are `Nacionality`, `International`, `Daytime/evening attendance`, and `Educational special needs`. And I measure what dropping them costs.

	CV PR AUC	Columns
Every raw feature	0.817	215
Engineered and selected	**0.819**	**84**

Sixty percent of the columns are gone, and the score went slightly **up**. The gain is too small to be real, and that is fine. The point is that cutting 131 columns costs nothing.

Dimensionality reduction



PCA is a way to squeeze many columns into a few combined ones that carry most of the information. Run on the nine number columns, it needs **seven of the nine** combined columns to keep 90 percent of the spread in the data. So there is almost nothing to compress here. That matches the low overlap the VIF check already showed.

That is an honest negative result, and last time I left it there. **A chart that no model ever uses is not really a method applied.** So PCA goes into Step 4 as a seventh model and has to earn its seat against the other six.

Step 4. Modeling and Comparison

I tuned seven models inside five-fold cross-validation on the training set. That means each model was tried on five rotating slices of the training data, with the dropout rate kept steady in every slice. To handle the mild imbalance I gave the rarer class more weight, a light touch that fits a 39/61 split. I did not make up fake extra dropouts. The test set is scored just once, at the very end. Each model's best settings are saved to `models/metrics.json`, so every row here can be rebuilt.

Model	CV PR AUC	Test PR AUC	ROC AUC	Recall	Precision	F1
XGBoost	**0.824**	0.828	0.857	0.746	0.665	0.703
Random forest	0.820	0.823	0.855	0.725	0.669	0.696
Logistic regression	0.819	0.822	0.851	0.750	0.670	0.708
Logistic regression + PCA	0.818	0.821	0.852	0.757	0.666	0.708
SVM	0.808	0.817	0.858	0.722	0.704	0.713
MLP (neural net)	0.752	0.753	0.798	0.669	0.667	0.668
Decision tree	0.723	0.699	0.762	0.683	0.601	0.639

PCA lands fourth, one thousandth behind plain logistic regression. Swapping the nine number columns for six combined ones costs almost nothing and buys almost nothing. That is a measured answer, and it is why PCA does not make the final cut. Each combined column is a blend of nine real columns. So a student flagged by it could never be told why in any way that means something.

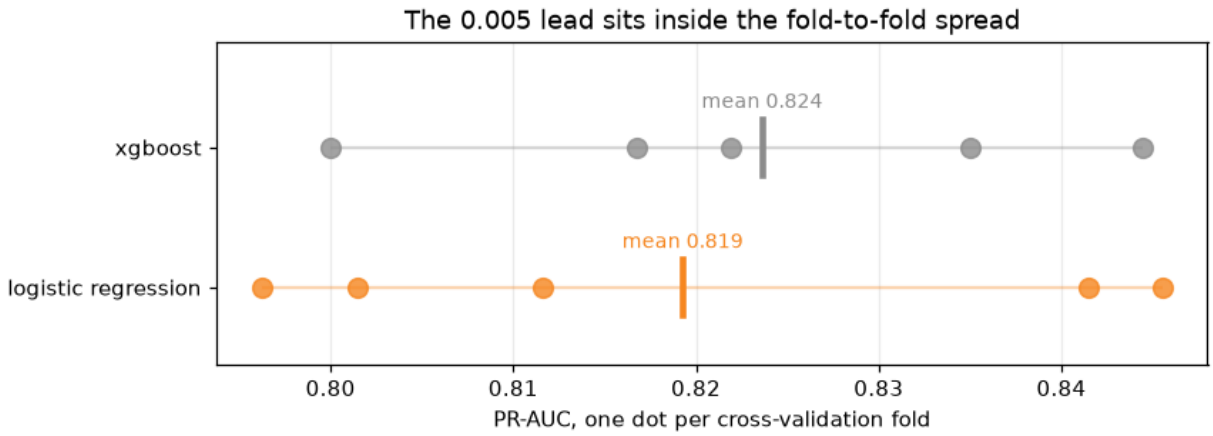
The neural network loses, below everything except the lone decision tree. That answers the question it was there to ask. Deep learning does not beat tree models, or a simple straight-line model, on data this small and this table-shaped.

Is the XGBoost lead real?

XGBoost leads by 0.005. Before trusting that lead, look at how much the five slices disagreed with each other.

Model	Mean	Std	Folds
Logistic regression	0.819	**0.020**	0.796, 0.841, 0.812, 0.801, 0.846
XGBoost	0.824	0.015	0.800, 0.835, 0.822, 0.817, 0.844

The gap is 0.005. The slices swing by 0.020. The lead is smaller than the normal wobble in the data. Rerun with a different random seed and the order could flip. So XGBoost is not better than logistic regression on this data. It is tied, and it just happened to land on top.



The chart makes the case. Each dot is one slice. The two ranges overlap almost completely, and the small gap between their averages vanishes inside that overlap.

The choice is logistic regression. Not because it scores highest. It does not. When two models are tied, the tiebreak should be something that matters. Here that is whether a student can be told why they were flagged. A logistic regression weight is a number a tutor can read out loud. A stack of 300 boosted trees is not.

The threshold nobody chose

Every recall figure above uses a cutoff of 0.5. **Nobody actually chose 0.5.** It is just the software default. On this data it happens to flag 44 percent of the students. So recall at 0.5 tells you about the default setting, not about the model.

Step 1 wrote down a business case and then never used it again. This is where it belongs. The cutoff is chosen on the training data, using the held-out slices from cross-validation, and applied to the test set only once.

Rule	Threshold	Recall	Precision	Flagged
Default 0.5	0.50	0.750	0.670	43.8%
Capacity 50%	0.38	0.838	0.609	53.9%
Capacity 45%	**0.45**	**0.796**	**0.644**	**48.3%**
Capacity 40%	0.50	0.750	0.670	43.8%
Capacity 35%	0.57	0.697	0.723	37.7%
Capacity 30%	0.64	0.630	0.768	32.1%

The 0.5 default is really a decision to staff for 40 percent of students. That is all it ever was, a staffing choice made by accident through a software default. Move the cutoff to 45 percent and recall climbs to **0.796** on the same model and the same data. The 45 percent is a target set on the training folds. On the unseen test set it lands at 48.3 percent flagged. The cutoff itself is 0.445, which the table rounds to 0.45.

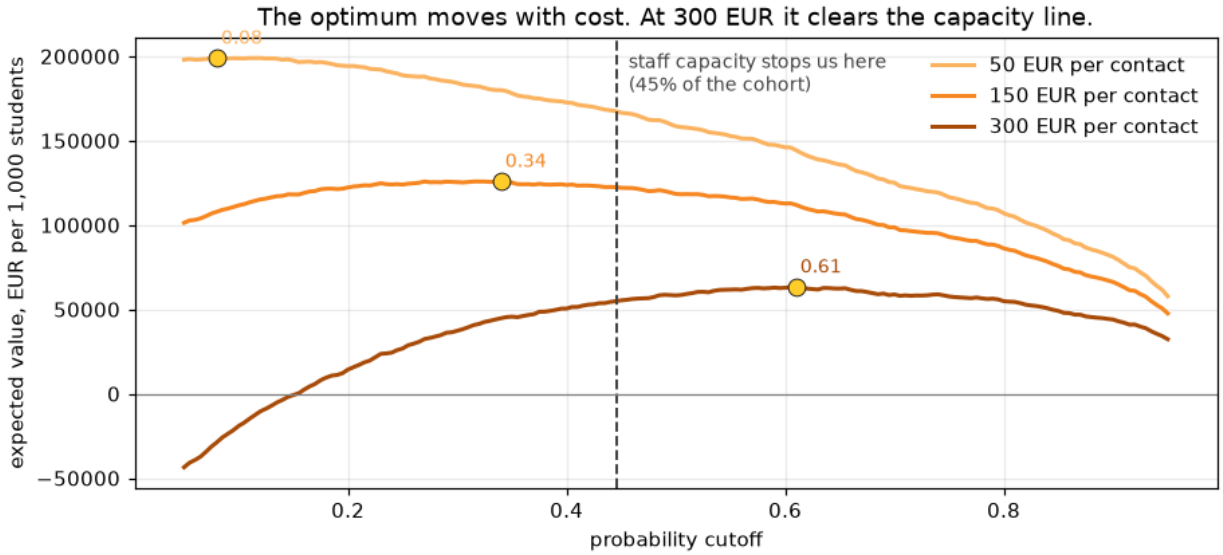
One thing to be clear about. Giving the rarer class extra weight makes it easier to flag, but it also throws off the meaning of the raw scores. So a score of 0.445 does not mean a student has a 44.5 percent chance of dropping out. It just means a place in a ranking.

That is fine, and it is worth saying why rather than glossing over it. Nothing later needs the score to be a true chance. The capacity rule just picks a spot in the ranked list, which only needs the order to be right. The value table below counts what actually happened at each cutoff rather than trusting the scores to be real chances. And Step 5

promises to never show the score to a student at all. These are ranking scores. Only the ranking is ever used.

Then the Step 1 numbers get a say.

Outreach cost	Best threshold	Flagged	Recall	Value per 1,000 students
50 EUR	0.08	93.1%	0.996	199,022 EUR
150 EUR	0.34	59.4%	0.870	125,289 EUR
300 EUR	0.61	34.4%	0.658	58,967 EUR



The economics say **flag broadly**. A saved student is worth about 630 euros on average, against 150 euros to reach one. So at the middle cost the model wants to contact 59 percent of the students. That is a strange thing for a targeting tool to say, and it is worth sitting with rather than hiding. When a save is worth four times what a contact costs, a wrong flag is cheap and a missed student is not. There is barely any narrowing-down left to do.

The chart caught me being sloppy, and the table had let me get away with it. I had written that staff time runs out long before the economics do. Look at the 300 EUR curve. It peaks at **0.61, higher than the capacity line**. At that cost the economics want *fewer* flags than the tutoring team could handle. So capacity is not the limiting factor at all.

So the honest version has a condition. Staff capacity is the limit at 50 and 150 EUR per contact. At 300 EUR the money runs out first. Which case the school is in depends on a number I do not have. Three rows of a table let me skate past that. The curve does not.

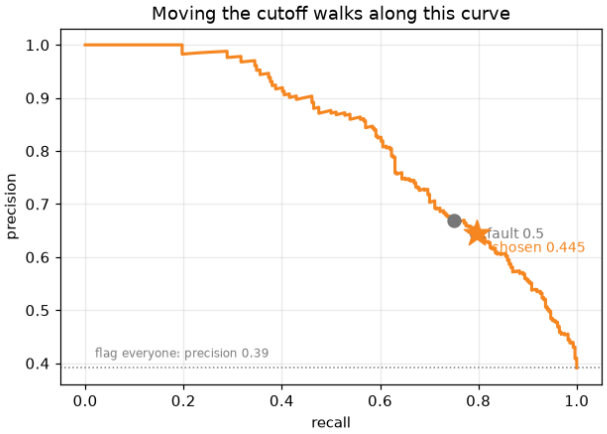
What holds either way. **The ranked list is the product. The yes or no flag is not.** A tutor working down a list from most at risk to least does not need a cutoff, and does not care which limit bites first.

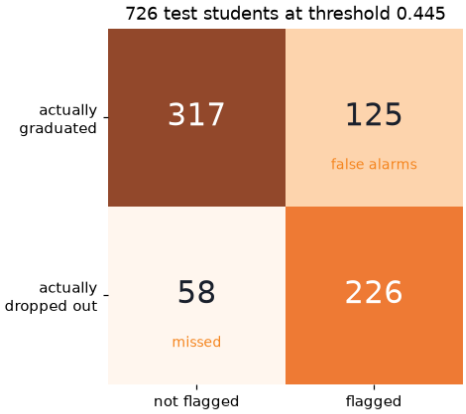
A cutoff is still needed to run the fairness audit and to report one honest number. So it goes at **0.445**, the capacity-45-percent point. This assumes staff capacity is the limit, which is the case at the low and middle outreach costs.

The cost of fitting to capacity instead of chasing the most profit is small, and that deserves a number. The rows above show the most profitable cutoff for each cost. At the chosen cutoff of 0.445 the value drops a little, to **171,942, 123,595, and 51,074 EUR**

per 1,000 students at 50, 150, and 300 EUR a contact. At the middle cost that is 1,694 below the best case of 125,289. So aiming the tool at the team's capacity rather than at the most profitable cutoff is very nearly free.

At the operating point





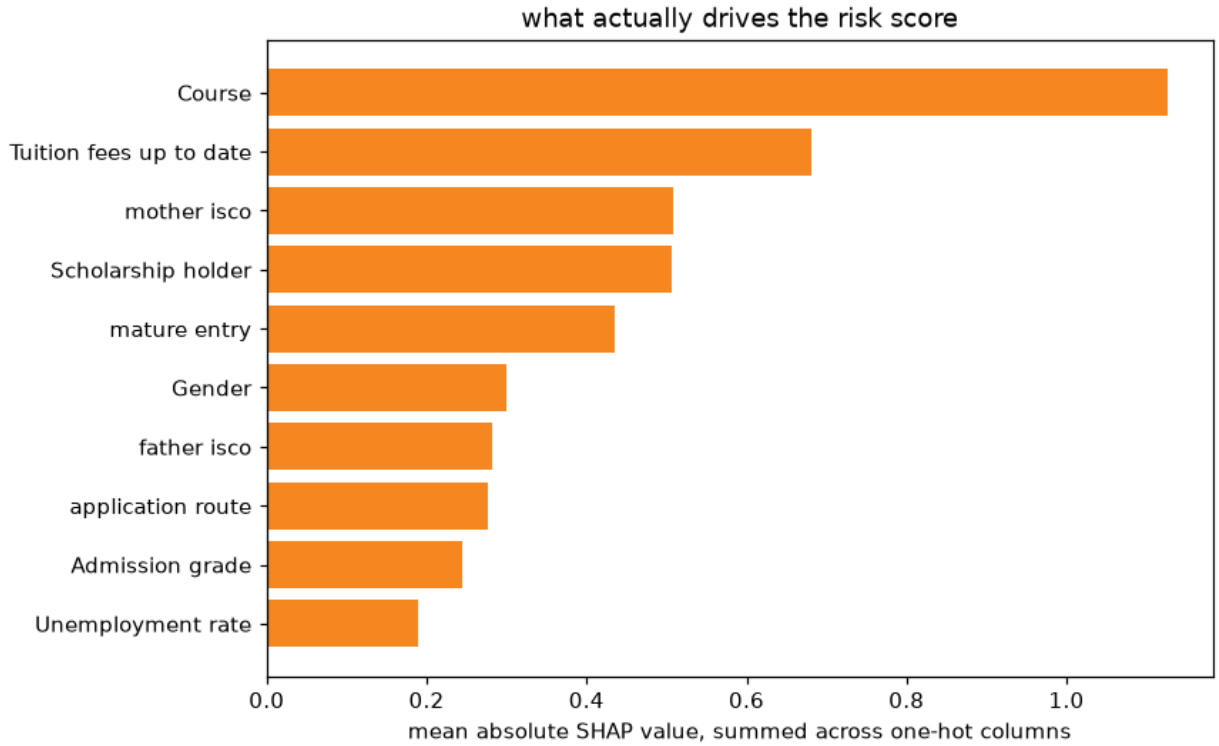
	Not flagged	Flagged
Actually graduated	317	**125**
Actually dropped out	**58**	226

226 leavers caught. 58 missed. And **125 graduates contacted who were never going to leave**. Those 125 are the real cost of this tool, and they are not free. A student pulled into a retention conversation they did not need has been told, in effect, that a system thinks they are failing.

Step 5 asks who those 125 students are. The answer is not spread evenly.

Step 5. Ethics, Bias, and Fairness

How the model decides



This chart was wrong the first time, and wrong in a way that was hard to see.

It drew one bar per column in the feature table. But `Course` is not one column. It is seventeen yes or no columns, one per course. So its importance got chopped into seventeen small bars, none big enough to notice. Meanwhile a single flag like tuition status kept all its weight in one bar. Features with many categories were being buried by the way they were encoded, and I was reading the result as if it meant something.

Here is the same chart with each feature's weight added back up onto the original feature.

Feature	Mean absolute SHAP
Course	**1.126**
Tuition fees up to date	0.680
mother isco	0.509
Scholarship holder	0.507
mature entry	0.436
Gender	0.301

(A second bug lived here too. If you hand the SHAP tool a plain array, it quietly picks a random sample of 100 rows to work from, with no fixed seed, so the numbers drift on every run. Tuition came out at 0.90 one run and 0.68 the next. The tool now uses all 2,904 rows. An explanation that changes when nothing else changed is not an explanation.)

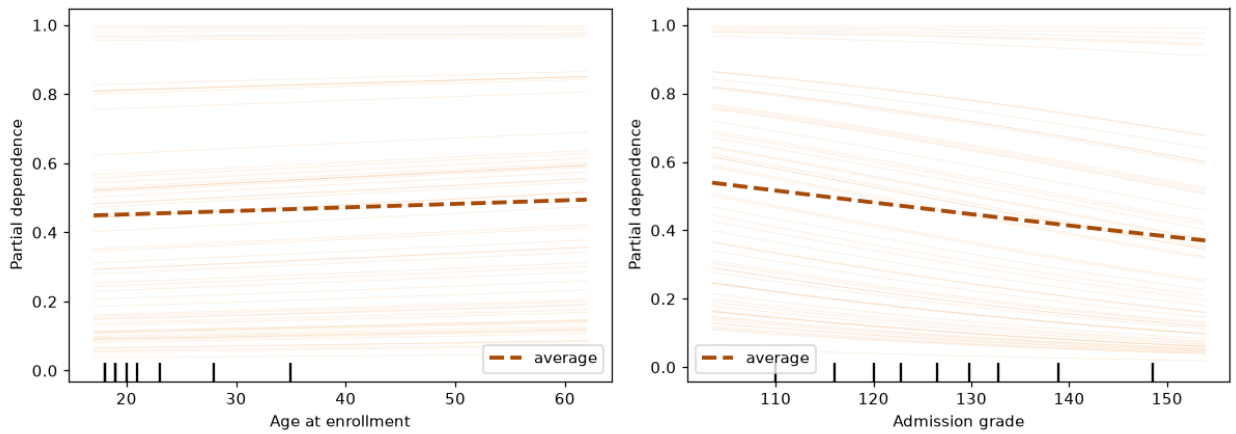
One caution on that ordering, because it is thinner than it looks. Mother's occupation reads 0.509 and holding a scholarship reads 0.507. That is two thousandths apart. In

Step 4 I refused to call a 0.005 lead real when the data wobbled more than that, and nobody has measured how much this SHAP ranking wobbles. So this report does not claim which of those two is higher. It does not need to. Course at 1.126 is far ahead of everything, and a parent's occupation sits well above admission grade at 0.245 either way.

This does not undo Step 3, where tuition led. Those earlier rankings, Cramer's V and mutual information, measure raw association one column at a time. SHAP measures something different. It measures how much the fitted model actually leans on each feature. Course leads on SHAP because its seventeen categories carry weight together that no single-column score can see. The two measures answer different questions, so they do not clash.

That changes what this project is about. The story I had was a money story about tuition, scholarship, and debt. It is still there and still strong. But the thing above it is **which course a student enrolled in**. And sitting level with the money signals is **what the student's mother does for a living**.

Neither is a warning a student can act on. Nobody chooses their mother's job, and by the time the model runs, the course is already picked. So the model is not mostly finding students in trouble. **It is largely finding students who started from further back.**



Risk climbs with age and falls with admission grade. The thin lines, one per student, run parallel. That is exactly what a simple straight-line model with no interactions should look like. This is a check passing, not a new finding. But it is worth showing, because an average line can hide a feature that pushes half the students one way and half the other.

Limits, stated honestly

- **Imbalance.** A 39 percent dropout rate is only mildly uneven. Extra weight on the rarer class handles it. PR-AUC is the lead measure.
- **Leakage.** The model uses only what is known at enrollment. Tuition status is the softer, watched case, and its cost was measured on the training data, not on the test set.
- **Unreadable weights.** This limit is here because it nearly ruined the project. Every explanation is read straight off the model's weights. A weight with the wrong sign becomes a sentence that tells a student the opposite of the truth. The rank check is now a standing guard. Six built-in overlaps for six category groups, and not one more.
- **Overfitting.** Training PR-AUC is 0.840 against 0.822 on the test set, a gap of 0.018. So the model learned the real pattern, not the noise.
- **Scope.** IPP-style school only.

The fairness audit, and the metric I was using wrong

The first audit used a test called demographic parity on all four groups. It found every one below the 0.8 line and concluded the model fails on all four. **That conclusion was too easy, and it was partly wrong.**

Demographic parity asks whether the model flags each group at the same rate. That is the right question when the groups do not really differ. It is the wrong question when they do. Students who owe money drop out at 76 percent, and the rest at 34 percent. Asking for equal flagging across that would be asking the model to ignore a 42-point difference it can plainly see. **That is not fairness. That is asking the model to be wrong on purpose.**

So the real-world gap goes into the table, right next to the model's gap. If the model's flagging gap is about the same size as the real outcome gap, the model is just **reporting** a difference that already exists. If the model's gap is bigger, the model is **making** the difference worse.

Attribute	Real gap in outcomes	Flagging gap (DP)	DI ratio	Error gap (EO)	Recall gap
Gender	0.238	**0.350**	0.496	0.290	**0.177**
Scholarship	0.318	0.500	0.171	0.385	0.283
Debtor	0.392	**0.369**	0.545	0.199	0.199
Age band	0.433	0.558	0.369	0.572	0.253

A quick guide to the table columns. "Flagging gap (DP)" is the demographic parity gap, the difference in how often two groups get flagged. The "DI ratio" is the disparate impact ratio, the flag rate of one group divided by the other, where anything below 0.8 is the usual warning sign. "Error gap (EO)" and "recall gap" measure fairness in the mistakes, not the flags. Equalized odds (EO) asks whether the model makes errors at the same rate for each group.

One note on the real-gap column. It uses the dropout rates on the 726 test students, so it can be set beside the model's test-set flagging gaps. Those rates differ a little from the full-data rates in Step 2. Gender reads 0.238 here against 0.259 there, because the split lands 154 dropouts among 288 test men and 130 among 438 test women.

Debt. Real gap 0.392, flagging gap 0.369. The model's gap is *smaller* than the one in the world. It is under-reporting a real difference, not inventing one. Its disparate impact ratio of 0.545 sits well below 0.8, and the old audit called that a failure. It is not. Debt is not a protected trait, and flagging debtors and non-debtors at the same rate is not something anyone should want.

Gender. Real gap 0.238, flagging gap **0.350**. The gap the model creates is *bigger* than the gap in the world. **The model is making it worse.** This is the one group here where demographic parity asks exactly the right question and gets back a bad answer.

Scholarship. Real gap 0.318, flagging gap 0.500. The model's gap is bigger here too, and by more than it is for gender. So the model amplifies this one. It does not just report it. What saves it is the same thing that saves debt. Scholarship is not a protected trait, and the heavier flagging tracks a real risk. Scholarship holders drop out at 14 percent against 48 percent for the rest. Its low disparate impact ratio of 0.171 looks alarming and is not.

Age band. Real gap 0.433, flagging gap 0.558. The model amplifies this one as well. Age is not like scholarship and debt. Age can be a protected trait, and its error gap of

0.572 is the largest in the table. This one is not settled, and the residual risk section comes back to it.

The harm, which is not the one I expected

Gender	Flag rate	Recall	False alarm
Male	0.694	**0.877**	0.485
Female	0.345	**0.700**	0.195

The model flags men twice as often as women. Their false alarm rate is more than double too. So a man who would have graduated fine is far more likely to be pulled into a conversation he did not need.

But look at recall. **0.877 for men. 0.700 for women.**

Three in ten women who go on to drop out are never flagged at all. For men it is closer to one in eight. The students this tool exists to reach are being missed, and they are missed more often if they are women.

That is the finding. Not "every group fails disparate impact," which was true and useless. The model is quietly worse at its actual job for women than for men. No amount of staring at flagging rates would have shown that. It shows up only when you stop counting flags and start counting **mistakes**.

Whether the model should see gender at all is a separate question, and the answer is not the obvious one. Dropping the gender column does not remove gender. The model still sees Course, the strongest thing it uses, and courses in this data are strongly split by gender. **Removing a protected trait from a model that keeps its stand-ins makes the bias harder to measure without making it any smaller.**

Mitigation, and what each one charges

Both methods aim for **equalized odds**, not demographic parity. Equalized odds means matching the error rates across groups. The harm here is an error gap, so the fix has to work on errors, not flags. Both methods use a fixed seed, so their numbers hold still between runs.

Approach	Recall	Precision	Accuracy	Flagging gap	Error gap	Recall gap (magnitude)
Baseline at 0.45	**0.796**	0.644	0.748	0.350	0.290	0.177
Threshold optimizer	0.609	0.783	0.781	0.100	0.026	0.026
Exponentiated gradient	**0.718**	0.685	0.760	0.131	0.027	**0.009**

Reading those numbers, my first instinct was to say the second method closes the recall gap *better*, 0.009 against 0.026.

I should not have said that, and Step 4 is the reason why.

In Step 4 I refused to hand XGBoost the win for a 0.005 lead, because the data swung by 0.020 and a gap smaller than that swing is not real. **Then I came here and quoted fairness numbers to three decimal places off a single 726-student split, without ever asking how much they wobble.** I had been careful in one place and careless in

another. So I went back and checked. I re-drew the test set 2,000 times with a fixed seed and re-measured each time. That is the bootstrap.

One note on signs, because the two tables read differently on purpose. The mitigation table above reports the recall gap as a **magnitude**. The bootstrap below reports it **signed, male minus female**. Baseline is +0.177 either way, so the two only part company on the mitigated rows, where 0.026 above becomes −0.026 below. The gap inverts after mitigation, and women end up caught very slightly more often than men. Both intervals span zero, so that direction carries no information at all. The fourth row compares the two magnitudes, which is why it lands at +0.008 rather than at the difference of the two signed numbers.

	Point estimate	**95% interval**	
Baseline recall gap	+0.177	[+0.082, +0.274]	**real**
Threshold optimizer recall gap	−0.026	[−0.142, +0.090]	indistinguishable from zero
Exponentiated gradient recall gap	−0.009	[−0.118, +0.101]	indistinguishable from zero
TO gap minus EG gap, on magnitudes	+0.008	[−0.057, +0.076]	**tied**
EG recall minus TO recall	**+0.109**	**[+0.075, +0.146]**	**real**

Each 95 percent interval is the range where the true value almost certainly sits. If that range includes zero, the number could just be chance. Three things fall out of the table.

The harm is real. The 0.177 gap never crosses zero across two thousand redraws. The model is worse at finding female dropouts, every time. This finding stands.

But the third decimal was just noise, and I was reading it. The recall gap rests on **130 female dropouts and 154 male ones**. It does not rest on all 726 test students. 726 is the size of the test set, not the size of the thing this number depends on. **So 0.009 is not better than 0.026. They are the same number in different clothes.** Catching myself making the exact mistake I had refused to make one notebook earlier was a humbling way to learn that care is not a mood you sit in. It is a check you run.

And exactly one difference survives the test. That is the recall cost, at +0.109, with a range nowhere near zero. Which means the choice was never really about fairness at all.

The verdict, on the one thing that survives

The two mitigations are tied on fairness. Both close a gap that was genuinely there. Neither is measurably better at it.

They are not tied on cost, and that is the whole decision.

The **threshold optimizer** drops recall from 0.796 to **0.609**. That is nineteen points. Look at how it got there. It closed the gap between men and women mostly by catching fewer people overall. So it bought fairness by missing more students of both kinds. Accuracy went *up* while the tool got *worse* at its actual job. That is a useful reminder of how little accuracy is worth on a problem shaped like this.

The **exponentiated gradient** drops recall to **0.718**. That is eight points. It reaches the same fairness, and it keeps eleven points of recall that the other method throws away. This second method is a training approach that builds a mix of models to meet the fairness target while giving up as little accuracy as it can.

Take the exponentiated gradient. Eight points of recall is still a real cost, roughly thirty fewer leavers caught per thousand students, and it should be written down plainly rather than buried. What it buys is that those thirty missed students are not loaded onto women.

What the fix costs, in the same euros Step 4 used. The Step 1 assumptions have to price this too, or the recommendation is a number nobody can check.

Per 1,000 students	Leavers caught	Students flagged	Value at 150 EUR a contact
Baseline at 0.445	311	483	123,595 EUR
Exponentiated gradient	**281**	**410**	**115,455 EUR**

A saved student is worth 630 EUR after the 3-in-10 uplift, and a contact costs 150. The reduction catches thirty fewer leavers but also flags seventy fewer students, so it claws back some of the loss on outreach it no longer spends. **The fairness fix costs about 8,000 EUR per 1,000 students.** That is roughly one euro in fifteen of the value the tool produces, and it is what the school is being asked to pay to have the tool work as well for women as it does for men. The number belongs on the table, not in a footnote.

The other option was to fix the gap by helping fewer people. That is not a fix.

One honest note on how this choice was made. I picked between these two by reading their scores on the **test set**. In Step 3 I caught myself settling the tuition question on the test set and moved it onto the training slices instead, and I was pleased with myself for it. This is the same act. It is easier to defend here, since it is the final comparison of two finalists at the very end. But by the standard I set for myself in Step 3, the stricter way is to choose the fix on the training data and touch the test set exactly once, only to report. I am naming it rather than hoping nobody notices.

Residual risk, and what I would tell the school

The model works, and it works on things students cannot change. Course comes first, far ahead of everything. A parent's occupation sits level with the money signals rather than below them. Tuition and scholarship, the money problems the school could actually do something about, sit around them.

That suggests a flag is the wrong shape for the help, and Step 4 reached the same conclusion from the economics. A ranked list, read from the top, with the reason attached, is something a tutor can work with. A yes or no at-risk label stamped on a student's record is something that follows them around.

There is a catch in that, and it is worth measuring rather than waving at. The shipped model is the exponentiated gradient, which makes a fair decision but not a score. Ranked on its own vote it collapses to five coarse bands, PR-AUC 0.617 against the base model's 0.822. So the shipped model cannot order the list. The order comes from the base model's score. That sounds like the recall gap sneaking back into the order a tutor works, but under the deployment I recommend it does not. The fair model flags 410 students per 1,000, and the team can staff 450, so the whole fair list gets contacted. That fit is not automatic. The unmitigated list flags 483 and overshoots the same 450 by

thirty-three, so it was the fairness fix that first pulled the list inside what the team can reach. The base score only sets the sequence they are worked in, a triage order, not a gate. Nobody is dropped for where the base model ranked them, so the fair set's recall gap of 0.009 governs, not the base model's 0.177. The seam bites in one case only. If the team cannot finish the list, the tail is dropped in base-score order and the gap comes back through the back door. So the design is to let the fair model choose who is on the list and the base model choose the order, and the fix holds as long as the list gets worked.

Gender is fixed. Scholarship and debt are not, and that is the right call. Debt the model under-states, and scholarship it amplifies, but neither is a protected trait and both track real risk. **The age gap is genuinely unresolved** and needs its own pass.

Four things I would insist on before this goes near a student.

1. **The score is never shown to the student as a number.** A person told they are 78 percent likely to fail has been handed a prediction, not help.
2. **If the score is built on something the school could change, the school should change it.** Tuition status is the second strongest signal, and a payment plan is a cheaper fix than a tutor.
3. **The recall gap gets re-measured at every intake.** It was 0.177 on this group of students, with a bootstrap range of roughly [0.08, 0.27]. That range is wide, because it rests on just 130 female dropouts. It will not stay fixed on its own, and one group of students is not enough to call it settled.
4. **The whole flagged list actually gets worked.** The fairness guarantee rests on the fair set of 410 fitting inside the 450 the team can staff. If the team stops short, the tail is dropped in base-score order and the recall gap comes back through the back door. The fix is only as strong as the resourcing behind it.

Generative AI

Step 5 produces SHAP values, and a tutor cannot read a SHAP value. `Tuition fees up to date: +0.68` is true and useless to the person who has to pick up the phone. Turning that into a sentence is the one thing a language model is genuinely good for here. I built it as the optional Step 9, and I built it the way the rest of the project works, against a baseline and behind a check.

There are two ways to turn a student's drivers into a note. A rules engine, a lookup table from each feature to a clause and an action, deterministic and unable to invent anything. Or a language model, handed the student's top SHAP contributions and asked for two plain sentences. The notebook builds both, and asks the honest question, what did the language model buy that the table did not.

The point is not that it calls an API. The point is the verifier. Step 5 promises three times that a student never sees a number, so the prompt forbids numbers, scoring words, any driver not in that student's own list, and gender, even though the model leans on gender at a SHAP value of 0.301. Then a cell checks that the prompt actually held. To prove the check does something, the same task runs twice, once with a naive prompt and once with the guarded one. The naive prompt failed all five students. Every note printed raw values and admission scores, four of them read a parent's occupation as a sign of low social class, one saying "lower socioeconomic status" outright, and the fifth named the student's gender. The guarded prompt passed all five. **A prompt is a request. The verifier is the control.**

The second use is the one the brief names directly, an auto-generated summary of the exploratory analysis. The model is handed the computed statistics rather than the data, writes the paragraphs, and then every number it wrote is checked against the numbers it was given. It invented none.

What the language model bought was fluency, and words for the slots the table leaves blank, the course code and the parent's occupation the table refuses to touch. What it also bought back is worth stating plainly. Three of the five guarded notes still turned a parent's occupation into a claim about familiarity with higher education, and one put it in exactly those words, "less familiarity with higher education". That is an inference the table declined to make and one the verifier is not built to catch. The verifier is necessary, not sufficient. And everything the model knows about which number matters, it knows because it was told, in a context block that took the rest of this project to earn. **The context is the work. The generation is the easy part.** The whole notebook runs, and renders, for anyone with no key, off a committed cache that carries every prompt and every response in plain text.

Reproducibility

```
notebooks/          01 to 05, one per step, run in order, outputs
committed
src/                data loader, path helpers, and the feature module
data/               raw (fetched, not committed), processed, and the
data dictionary
models/             preprocessor, the Step 4 model, the shipped model,
the threshold,
                    and full metrics with hyperparameters
reports/            this report and its figures
presentations/      the two Step 6 decks, technical and business, as
pptx and pdf
.python-version     3.13, read by uv
requirements.txt    exact pins, read off the environment that produced
these outputs
```

The two presentations live in the repository rather than in this document. `technical.pdf` is twelve slides for peers and walks the same four mistakes this report does. `business.pdf` is ten slides for the people who decide, and it leads with money and with the one choice only the school can make. Every number on both is copied from an executed notebook cell.

```
./setup.sh                  # uv venv on Python 3.13, install pinned
deps, fetch dataset
source .venv/bin/activate
./run_notebooks.sh          # my own script, runs 01 to 05 in order and
saves the outputs
```

I wrote `run_notebooks.sh` because re-running five notebooks by hand is a thing I got wrong more than once. You have to run them in the right order, from a clean start, and remember to save each one. The script does all of that in one command. It also locks the project to its own Python, refuses to fall back to the system one, and checks the installed package versions against the saved list before it runs anything.

The order is not optional. Notebooks 04 and 05 read files that 03 and 04 write. Run them out of order and nothing crashes loudly. They quietly run against stale files, which is worse.

Every number in this report comes from running the notebooks in order against the real data with fixed seeds. **I found and closed three sources of silent drift.** All three are worth naming, because not one of them announced itself. In each case the code ran fine to the end and simply wrote down a different answer.

- **The SHAP tool.** Handed a plain array, it quietly picks a random sample of 100 rows, with no fixed seed. Tuition status came out at 0.90 on one run and 0.68 on the next.
- **The exponentiated gradient's predict step.** It returns a random draw from a *mix* of models, not one fixed model. From the same fitted object, the recall gap read 0.012 on one call and 0.001 on the next.
- **The environment itself.** At one point `requirements.txt` was rewritten from versions read off a different machine. That pinned five packages *behind* the ones that had produced every number here. Installing it would have downgraded pandas, numpy, scikit-learn, scipy, and matplotlib, run cleanly, and quietly disagreed with this report.

The first two are seeded now. The third is checked on every run.

Two models are saved, on purpose. `model.joblib` is the plain logistic regression, and every number in Step 4 is read off it. `model_shipped.joblib` is the one this report recommends deploying, the same model under the equalized-odds constraint, and it carries the fitted preprocessor with it because the reduction was fit on the encoded matrix rather than on raw columns. `metrics.json` carries a `shipped` block naming which is which, along with the recall before and after, the bootstrap interval on the gap both before and after the fix, and what the fix costs. The after-interval is the one that matters, because it is what shows the remaining 0.009 is indistinguishable from zero. Without it a reader of `models/` would have to take that on trust. Saving only the first would have left `models/` describing a system Step 5 says should not be deployed.

Everything else is seeded at 42. That covers the split, the cross-validation slices, mutual information, PCA, all seven models, both fairness fixes, and the Step 5 bootstrap. The notebooks are committed **with their outputs**, so every table and chart here can be checked on GitHub without running anything.

Conclusion

The project delivers a working early-warning model for student dropout. It uses only what is known at enrollment. On its own it catches **79.6 percent** of true dropouts at a cutoff chosen to match the school's real staffing capacity. The version I recommend shipping adds the Step 5 fairness fix, which trades that down to **71.8 percent** to close the gender recall gap from a real 0.177 to 0.009, a gap the bootstrap can no longer separate from zero. And it stays simple enough to explain its own decisions.

It also produced four lessons, and each one came from catching a mistake I had made.

A feature that adds no information cannot help a model, but it can still break it. And it breaks it in the place that matters most, which is the model's ability to say why.

The model's strongest signals are things students cannot change. These are course and parental occupation. That is not a bug to patch out. It is what the data says, and a school deploying this should know it before it starts flagging people.

The harm was not where the standard fairness test was pointing. Demographic parity said all four groups failed. The truth was messier. The model under-stated only debt. It widened the gaps for scholarship, gender, and age. Gender was the one that

mattered, because gender is protected. The damage showed up as a recall gap that no flagging-rate test could see.

And rigor does not carry over on its own. In Step 4 I refused to call a 0.005 lead real when the data swung by 0.020. Then in Step 5 I quoted fairness numbers to three decimal places off just 130 female dropouts, as if the wobble had stayed behind in the other notebook. It had not. The bootstrap that caught this did not overturn the finding. The recall gap is real. But it did overturn the *reason* I gave for choosing one fix over the other. Skepticism is not a stance you adopt once and carry around. It is a check, and you have to run it everywhere it applies.

A dropout model is not a scoreboard. It is a decision about which students get help. The right way to use this one is as a ranked prompt for a support team, checked by a person, aimed at starting a conversation early enough to matter.

References

- Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). *Predict Students' Dropout and Academic Success* [Dataset]. UCI Machine Learning Repository. DOI 10.24432/C5MC89. License CC BY 4.0.
- Martins, M. V., Tolledo, D., Machado, J., Baptista, L. M. T., & Realinho, V. (2021). Early prediction of student's performance in higher education: a case study. *Trends and Applications in Information Systems and Technologies* (WorldCIST 2021), AISC vol. 1365, Springer, pp. 166–175. DOI 10.1007/978-3-030-72657-7_16.
- Funding: SATDAP, Capacitação da Administração Pública, grant POCI-05-5762-FSE-000191, Portugal.

Predicting Student Dropout, with a Fairness Audit

Four mistakes. Each one changed an answer.

Jeryl Donato Estopace

PGD AI/ML capstone, Pillar 5

UCI, Predict Students' Dropout and Academic Success. 4,424 students, Instituto Politecnico de Portalegre

The frame

3,630
students, after 794 with no settled outcome removed

39.1%
drop out. Mildly uneven, handled with class weights

12
curricular columns dropped, they leak

The task is a yes-or-no call between Dropout and Graduate, and the Enrolled rows have no settled outcome, so they cannot teach the model.

The twelve curricular columns predict dropout almost perfectly, because a failing student is already leaving. So the model sees only what is known at enrollment.

PR AUC and ROC AUC choose the model, two scores that hold at every cutoff.

Recall judges the deployed system, once a cutoff exists. Recall is the share of real dropouts caught, set by the model and the cutoff together.

Accuracy is reported and not trusted. Call everyone a graduate and you are right 61 percent of the time and catch nobody.

The gaps were there before the model

Dropout rate by group, against an overall 39.1 percent. Notebook 02.

Group	Dropout rate	n
Men	56.1%	1,249
Women	30.2%	2,381
No scholarship	48.4%	2,661
Scholarship holder	13.8%	969
Debtor	75.5%	413
Not a debtor	34.5%	3,217
Age 24 to 30	66.5%	499
Age 17 to 20	26.1%	2,080

94.0%

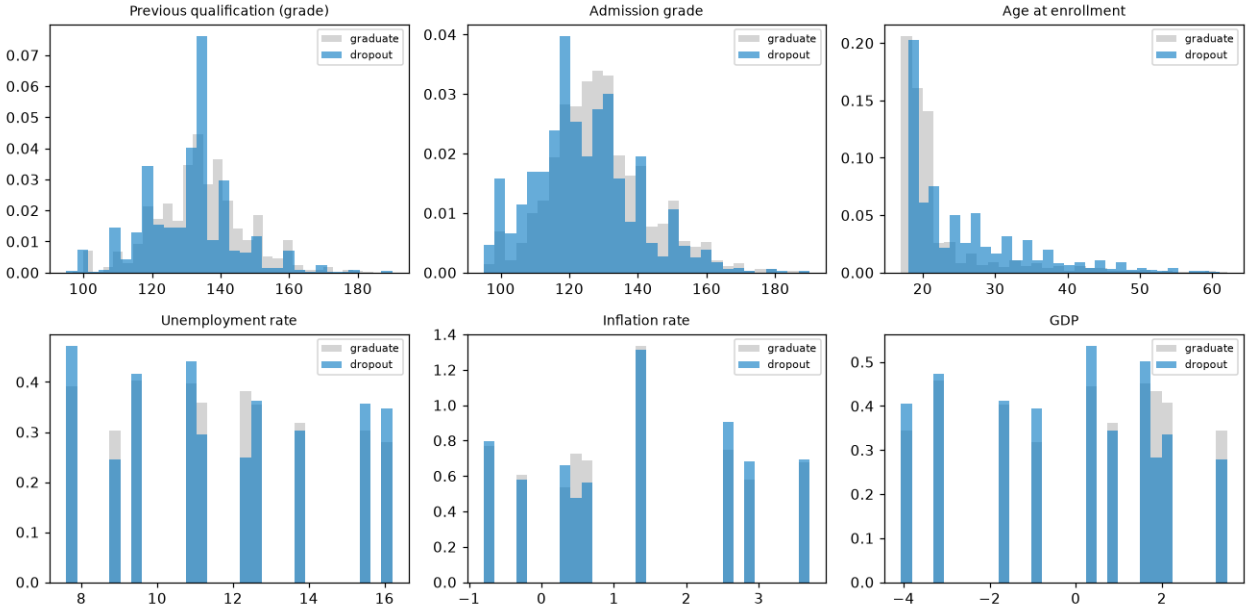
Dropout rate for the 486 students not up to date on tuition, against 30.7 percent for those who are.

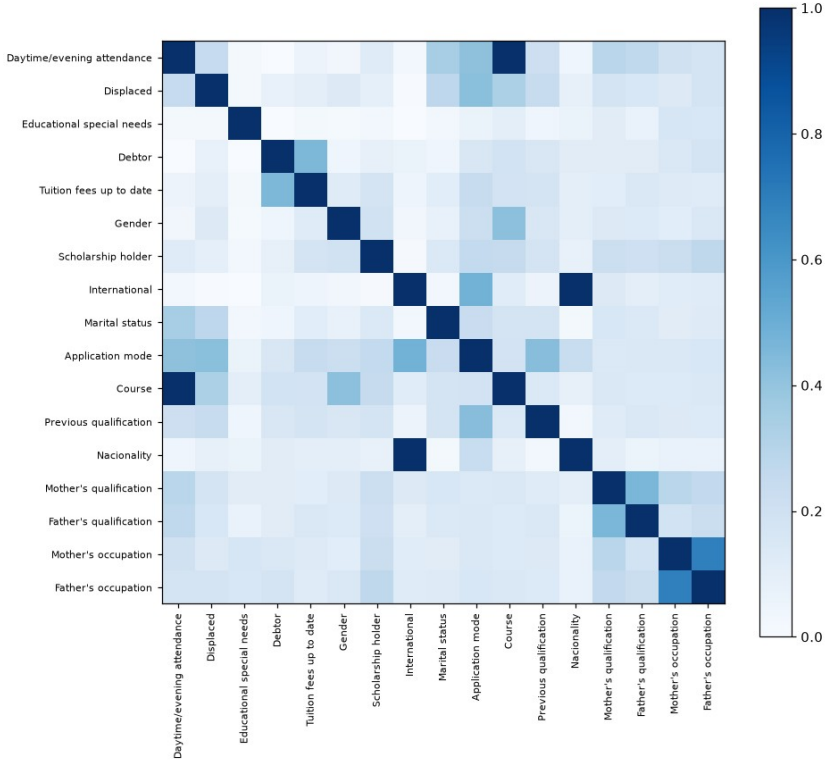
A student who has stopped paying is often already leaving, so this feature nearly is the outcome, and it is kept but watched, because removing it costs about five points of PR-AUC.

These group rates come back on slide 10, and they are the reason the fairness audit had to be rewritten.

What predicts dropout

The right test for each feature type. Mann-Whitney U on the numbers, chi-square and Cramer's V on the categories.





Age has the strongest link at p = 9.2e-65, because older entrants drop out more, and admission grade and previous qualification grade follow, while unemployment and inflation show no real difference at p = 0.75 and 0.21.

Tuition 0.437, Course 0.346, application mode 0.330 lead on Cramer's V.

Every VIF is under 2. VIF spots overlap in the six numbers only.

The matrix shows two pairs sitting at a perfect V = 1.000. I noted that and moved on.

1 The feature that broke the coefficients

I built a socioeconomic pressure score, one point each for owing money, falling behind on tuition, and holding no scholarship, and it ranked first on mutual information ahead of every raw column in the file.

Then the model turned around and said that owing the school money makes a student less likely to drop out.

```
pressure − Debtor + Tuition + Scholarship = [2]
```

It takes one value, exactly 2, on all 2,904 training rows, because the score was three columns the matrix already held.

Design matrix	Old	Fixed
Columns	216	84
One-hot blocks	9	6
Exact linear dependencies	19	6

Slide 4 said overlap was fine because VIF said so. But VIF only reads the six numbers, so it never saw the category columns and never saw a feature built after it ran.

The model and the data are the same, and the only change is whether the score sits in the matrix.

Coefficient	Score in	Score out	Real rate
Debtor	−0.465	+1.006	76% vs 34%
Scholarship holder	+0.138	−1.344	14% vs 49%
Tuition fees up to date	−1.480	−2.913	31% vs 94%

Two signs flip, because the model reads owing money as protective and a scholarship as a risk, when the data says the opposite.

Accuracy never moved, because the score carried no new information. It only made the model unreadable, and every Step 5 explanation is read off these coefficients.

A feature that adds no information cannot help a model, but it can destroy its ability to explain itself.

Features that replace, instead of repeat

A feature built from columns that stay in the matrix adds nothing to a linear model, so it earns its place only by replacing its sources.

The same rank check caught two more repeats, both already flagged at V = 1.000 and both waved past.

International equals (Nacionality != 1) on 100 percent of rows, which is the same column twice.

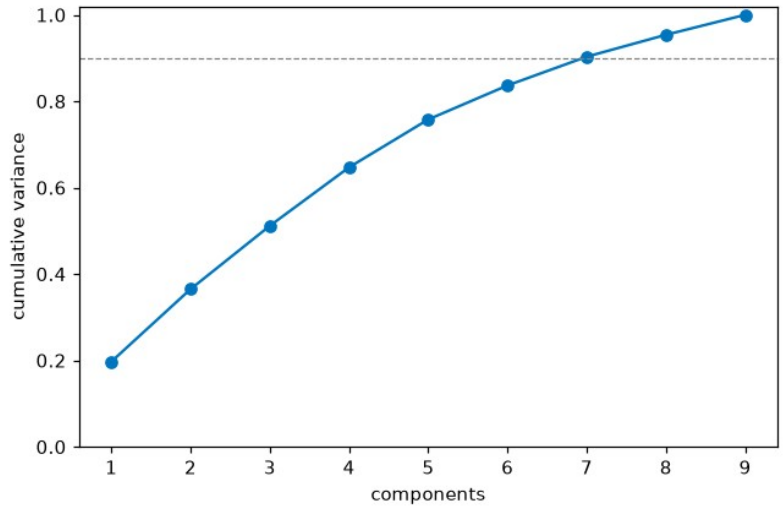
Daytime attendance is decided entirely by Course, because zero of the 17 courses run both a day and an evening class.

Five engineered features replace their sources and four weak columns come out, and then the cost is measured on training slices rather than assumed.

mature entry lands sixth at 0.0534, below the raw age column it bends, because it reads the whole age signal, so a cutoff drawn from it ranks lower, as it should.

Feature set	CV PR-AUC	Columns
Every raw feature	0.817	215
Engineered and selected	**0.819**	**84**

215, not the 216 on the last slide. The difference is the pressure score, now gone.



PCA needs seven of its nine parts to reach 90 percent of the variance, so there is nothing to compress, which matches the low VIF, and it rides into Step 4 as a seventh model rather than a chart nobody uses.

Sixty percent of the columns are gone and the score went slightly up, so the result is 131 columns cut for free.

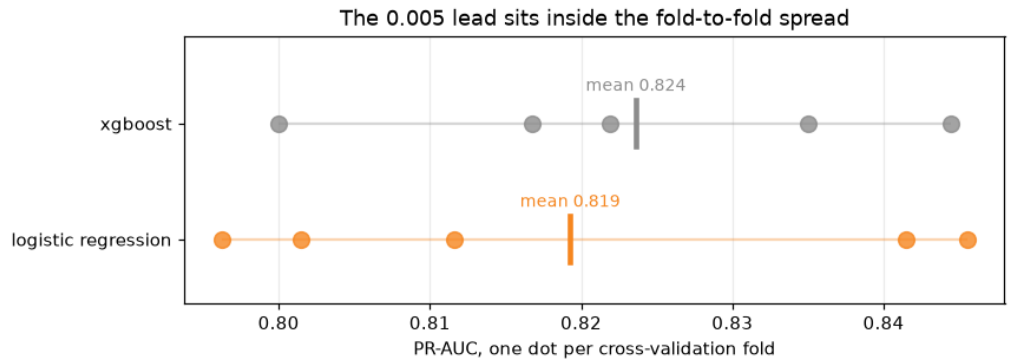
Seven models, and whether the lead is real

Tuned by cross-validation on five held-out slices of the training set, with the test set scored once.

Model	CV PR-AUC	Test PR-AUC	ROC-AUC
XGBoost	0.824	0.828	0.857
Random forest	0.820	0.823	0.855
Logistic regression	**0.819**	**0.822**	**0.851**
Logistic regression + PCA	0.818	0.821	0.852
SVM	0.808	0.817	0.858
MLP (neural net)	0.752	0.753	0.798
Decision tree	0.723	0.699	0.762

Logistic regression ships, and not because it scores highest. When two models tie, the tiebreak that matters is whether a student can be told why they were flagged.

PCA lands fourth, a thousandth behind, and a component is a blend of nine columns, so it could never explain a flag in a sentence that means anything.



The gap is 0.005. Slices swing by 0.020.

Each dot is one slice, and the two ranges overlap almost completely, so the gap between the means disappears inside them.

XGBoost is not better than logistic regression here but only tied with it, and it happened to land on top.

A gap smaller than the measured noise is not a gap, and this deck has to keep that standard everywhere.

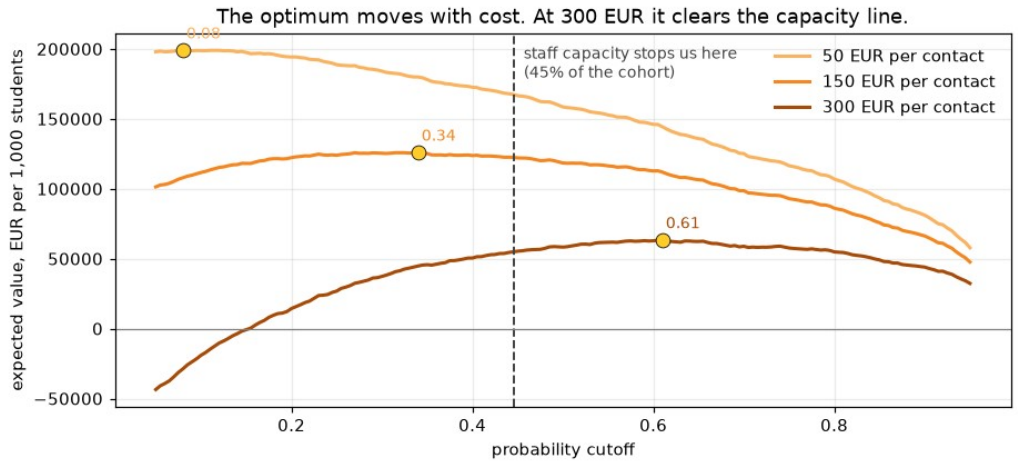
The cutoff nobody chose

Recall depends on the model and the cutoff together rather than the model alone, and the default cutoff is 0.5, which nobody chose.

Rule	Cutoff	Recall	Precision	Flagged
Default 0.5	0.50	0.750	0.670	43.8%
Capacity 50%	0.38	0.838	0.609	53.9%
Capacity 45%	**0.45**	**0.796**	**0.644**	**48.3%**
Capacity 40%	0.50	0.750	0.670	43.8%
Capacity 35%	0.57	0.697	0.723	37.7%

The default is capacity 40 percent. It is a staffing decision made by accident, by a library default, and moving it to 45 percent climbs recall to 0.796 on the same model and the same data.

Outreach cost	Best cutoff	Flagged	Value per 1,000
50 EUR	0.08	93.1%	199,022 EUR
150 EUR	0.34	59.4%	125,289 EUR
300 EUR	0.61	34.4%	58,967 EUR



The chart caught me being sloppy where the table had not. I had written that staff time runs out long before the economics do.

The 300 EUR curve peaks at 0.61, tighter than the capacity line.

Staff capacity binds at 50 and 150 EUR per contact, but at 300 EUR the money binds first, and the economics want fewer flags than the team could handle.

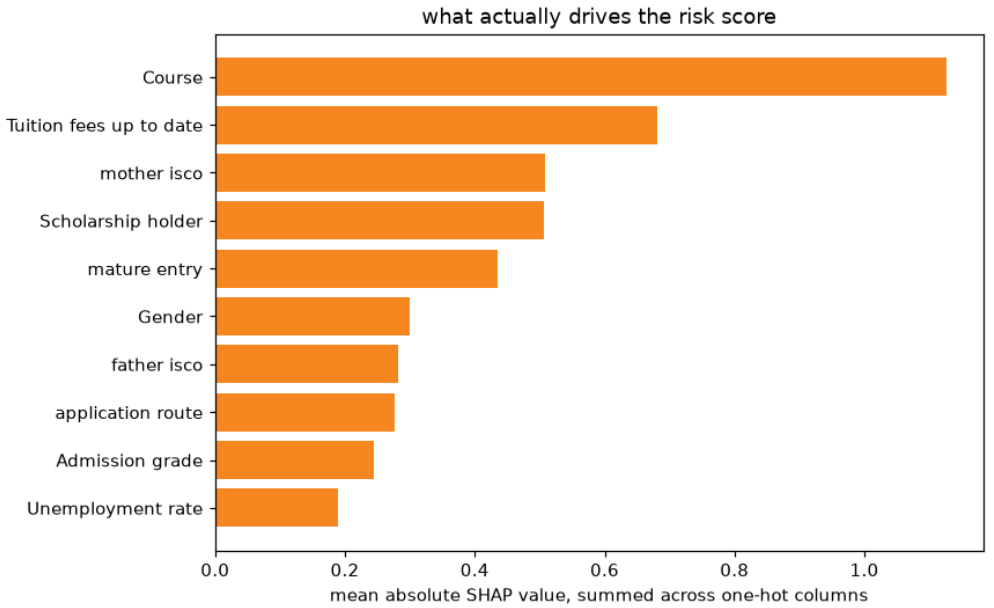
At the deployed 0.445 the value holds, 171,942 / 123,595 / 51,074 per 1,000. At 150 EUR that is 1,700 below the best case, so capacity is nearly free.

The ranked list is the product and the binary flag is not, and the cutoff goes at 0.445 to audit fairness and report one honest number.

2 SHAP ranked columns, not features

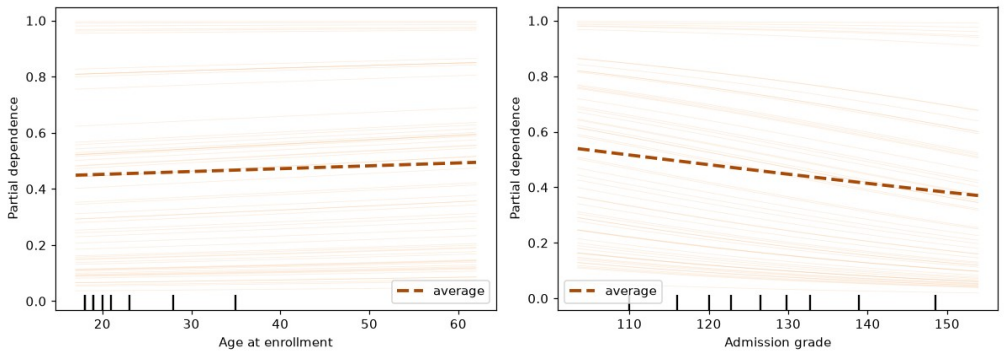
The first SHAP chart plotted one bar per column of the design matrix, but Course is not one column and instead seventeen of them, so its importance split into seventeen small bars while a single flag kept all its weight in one place.

Old, per encoded column		Fixed, summed by feature	
flag__Tuition fees up to date	0.680	Course	1.126
flag__Scholarship holder	0.507	Tuition fees up to date	0.680
flag__mature entry	0.436	mother isco	0.509
flag__Gender	0.301	Scholarship holder	0.507
cat__Course_9500	**0.284**	**mature entry**	**0.436**
num__Admission grade	0.245	Gender	0.301



cat__Course_9500 is the proof. The largest fragment of the strongest feature sat fifth, behind four flags, and mother's occupation did not appear at all. A second bug hid here too, because unseeded 100-row sampling drifted from 0.90 to 0.68 between runs, until the fix read all 2,904.

Course sits far ahead at 1.126, and where raw association put tuition first, inside the model course leads. Neither a course nor a parent's job is something a student can act on, and the model mostly finds students who started further back.



Prediction vs one feature. The lines run parallel, a check passing rather than a finding.

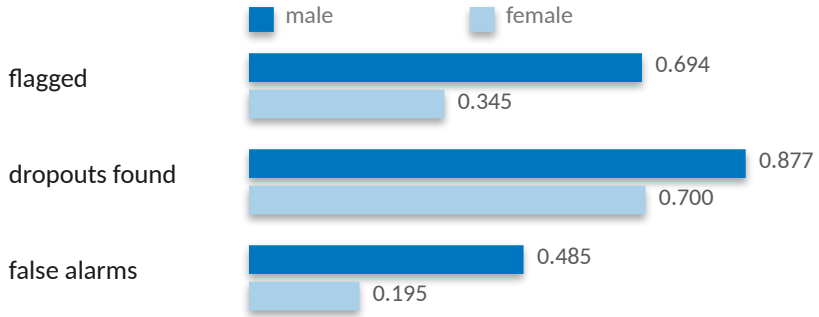
3 Equal flagging asked the wrong question

Debtors drop out at 76 percent against 34, so demanding equal flagging is demanding the model be wrong on purpose. The real-world gap sits next to the metric, so that a flagging gap near the outcome gap reports a disparity, and a bigger one makes one.

Attribute	Real gap	Flagging gap	DI ratio	Error gap	Recall gap
Gender	**0.238**	**0.350**	**0.496**	**0.290**	**0.177**
Scholarship	0.318	0.500	0.171	0.385	0.283
Debtor	0.392	0.369	0.545	0.199	0.199
Age band	0.433	0.558	0.369	0.572	0.253

Debtor. Real gap 0.392 and flagging gap 0.369, so the model's disparity is smaller than the world's. Its DI ratio of 0.545 sits below the 0.8 line the old audit called a failure, but it is not one.

Gender. Real gap 0.238 and flagging gap 0.350, so the gap the model creates is bigger than the gap in the world, and that is amplification.



3 in 10

of the women who go on to drop out are never flagged at all, and for men it is closer to one in eight. It appears in no flagging-rate metric, and only once you count errors instead of flags.

These gaps use the 726 test students, a little below the full-data rates on slide 3. The finding is not that every group fails the equal-rate test, which was true and useless.

4 Rigor does not generalize on its own

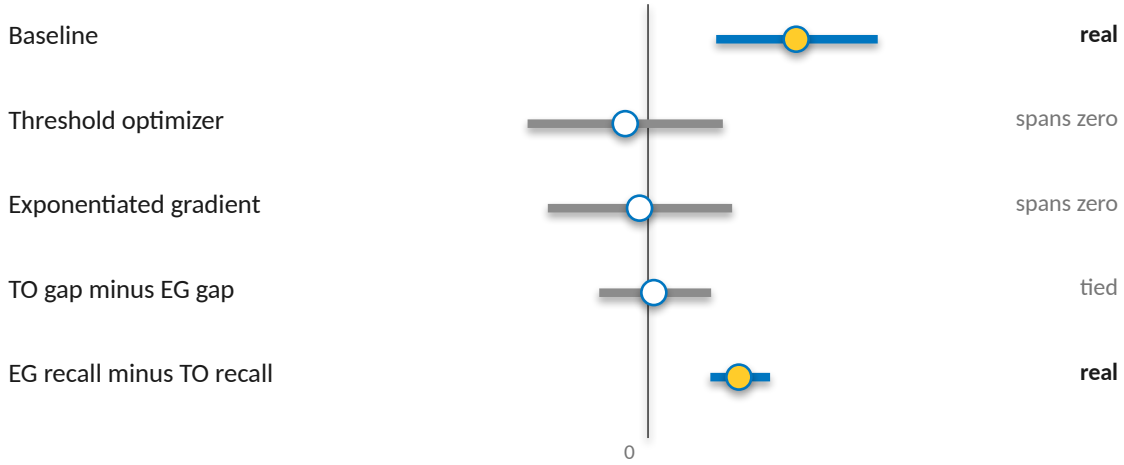
Step 4 refused to call a 0.005 lead real against a 0.020 swing, and then I quoted fairness numbers to three decimals off one 726-row split without asking what the noise was. So I went back and asked, with 2,000 seeded bootstrap resamples.

```
male      n=288    dropouts 154
female    n=438    dropouts 130
```

Women are the majority of the test set and the minority of its dropouts, so the recall gap rests on 130 students rather than 726.

Approach	Recall	Accuracy	Recall gap (magnitude)
Baseline at 0.45	0.796	0.748	0.177
Threshold optimizer	0.609	0.781	0.026
Exponentiated gradient	**0.718**	**0.760**	**0.009**

95% intervals, 2,000 resamples. The top three are the recall gap itself, male minus female. The bottom two compare the two mitigations against each other.



The harm is real. The 0.177 gap never crosses zero across two thousand resamples.

0.009 is not better than 0.026. They are the same number wearing different clothes.

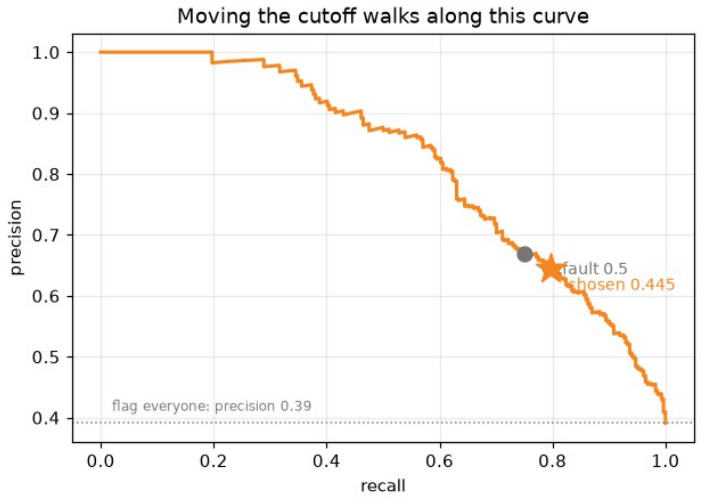
One difference survives, and it is cost. It is eight points of recall against nineteen.

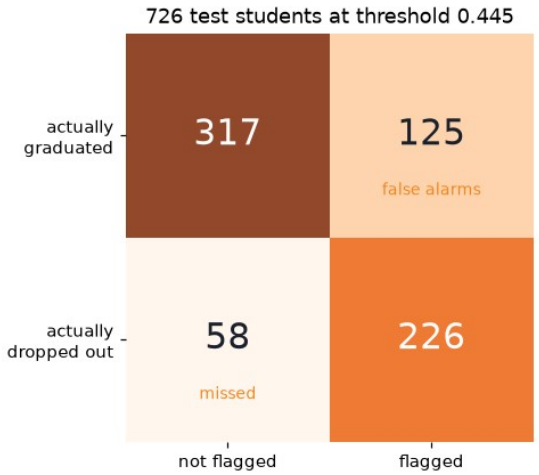
One interval clears zero on the harm, and one clears it on the cost. Everything between them is a tie. The sign flips after mitigation, so the mitigation table reports magnitudes and this one is signed.

The threshold optimizer closed the gap largely by catching fewer people. Accuracy went up while the tool got worse at its job. Fixing a gap by helping fewer people is not a fix.

The verdict, and what it costs

726 test students at the operating cutoff of 0.445.





226
leavers caught

58
missed

125
graduates contacted who were never going to leave

Recall 0.796 before the fix at a cutoff chosen against staffing, on enrollment-time data only. The shipped system adds the exponentiated-gradient fairness fix, recall 0.718, gender gap from a real 0.177 to 0.009, which spans zero.

Train PR AUC 0.840 against test 0.822, a gap of 0.018. The model learned the pattern, not the noise.

Gender is mitigated. Scholarship and debtor are deliberately not, because neither is a protected trait and both track real risk. Age band is unresolved, error gap 0.572, and it needs its own pass.

The fair model picks the list, the base model orders it, and the fair set of 410 fits capacity.

Those 125 are the real cost of this tool. A student pulled into a retention conversation they did not need has been told, in effect, that a system thinks they are failing.

A dropout model is not a scoreboard but a decision about which students get help, and the right way to use this one is as a ranked prompt for a support team, checked by a person.

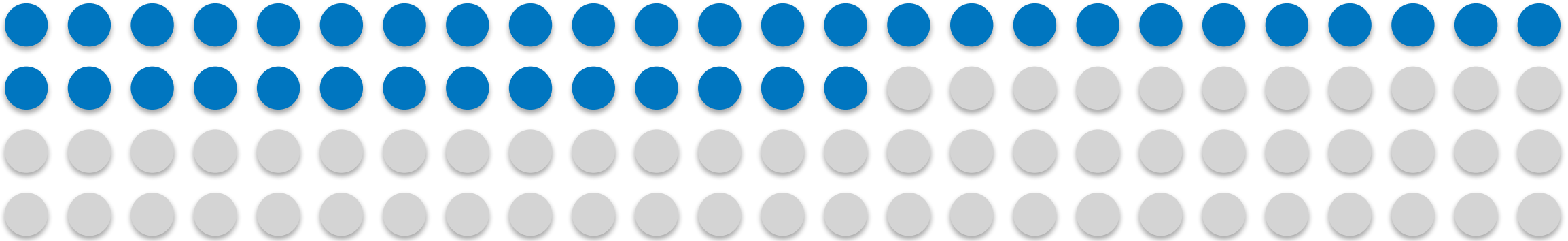
Finding the students who will leave, while there is still time to help

An early warning on day one. And one decision only the school can make.

Jeryl Donato Estopace

PGD AI/ML capstone, Pillar 5
Built on 4,424 student records from Instituto Politecnico de Portalegre

391 students in every 1,000 leave before they finish



Each dot is ten students, and the filled ones are the ones who do not graduate. The rate is 391 in every 1,000 among the 3,630 whose outcome is settled, once the 794 still enrolled are set aside.

Every dropout costs three parties at once, because the student loses years and fees while the school loses tuition and the country loses a graduate it had already half paid for.

The school knows this is happening, and what it cannot see is which students, because that only becomes clear when the first exams come back, by which time half the year is gone and so, usually, is the student.

The question is not whether to help, but who to help first, and when.

2,100 EUR

the tuition that one saved student pays back over a three-year degree, which is a deliberately low estimate because the real loss is larger.

What the tool does, on the day a student enrolls

It reads the enrollment form, and nothing else.

It knows the student's age and course, whether they hold a scholarship, whether the fees are paid, and what a parent does, and all of it is known before the first class.

It does not look at grades, attendance, or coursework.

That was on purpose and it costs some accuracy, because a tool that waited for first-term results would be more certain and also useless, since by then the student has already decided.

It hands the tutoring team a ranked list.

The students most at risk sit at the top, and it is a starting point for a conversation twelve months early rather than a number ever shown to a student.

12 months
of warning, instead of none

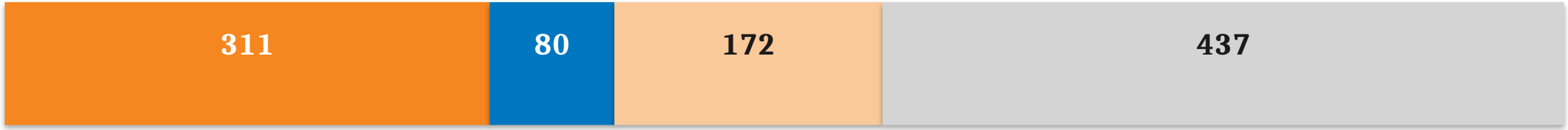
8 in 10
of the students who go on to leave are on the list

1 in 2
students are flagged, sized against what the tutoring team can handle

The tool does not decide anything, and all it does is decide what a person looks at first.

Out of every 1,000 students

Scaled from 726 real students the tool had never seen. It was tested once, on data held back from the start.



caught	missed	contacted for nothing	correctly left alone
leavers the team reaches in time	leavers the tool does not find	graduates who were never leaving	the team never hears about them

Of the 391 who leave, the tool finds 311.

Eighty are missed, which is the honest number and will never be zero, because no tool that runs on an enrollment form finds everyone.

The 172 contacted for nothing is the cost worth arguing about, and slide 6 does.

483

students flagged, about one in two, which is more than the team can staff, and the fairness fix on slide 8 is what brings the list back within reach.

Every number on this slide came from students the tool had never seen.

The return

Three assumptions, all stated, all careful. Change any one and the answer moves. That is why the curve is here, not just the number.

A saved student is worth about 2,100 EUR.

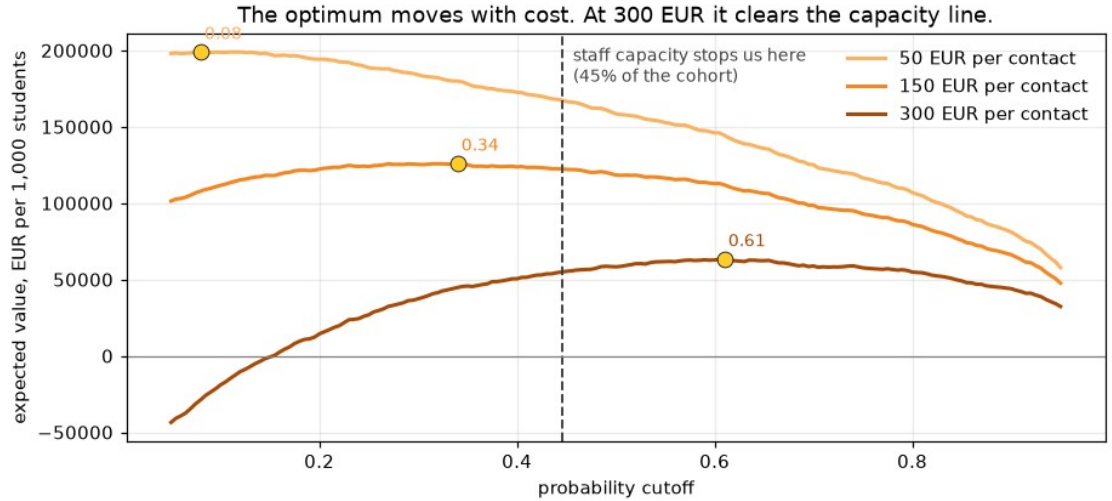
Three years of public tuition, and nothing else counted.

Reaching out works about three times in ten.

So a student the tool finds is worth about 630 EUR on average rather than 2,100.

A conversation costs somewhere between 50 and 300 EUR.

The school knows this number and I do not, so all three are shown.



A conversation costs	Best case	At 1 in 2
50 EUR	199,022 EUR	171,942 EUR
150 EUR	**125,289 EUR**	**123,595 EUR**
300 EUR	58,967 EUR	51,074 EUR

Value per 1,000 students. Aiming at the team's capacity instead of the most profitable cutoff costs only about 1,700 EUR at a 150 EUR contact, so it is nearly free, and slide 8 is the thing that is not.

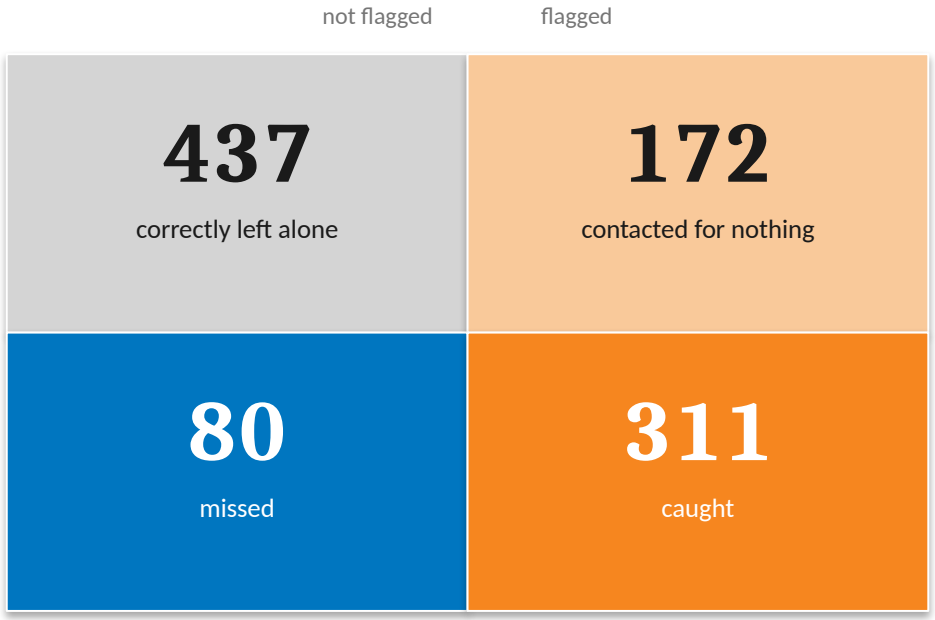
At 150 EUR a conversation, a save returns about four times what a contact costs, so the money says reach out broadly and there is barely any sorting left to do.

The price, and who pays it

172 in every 1,000 are contacted who were never going to leave.

The cost is not paid in euros but in a student pulled into a conversation they did not need, and told in effect that a system thinks they are failing.

If the tool flags fewer students then fewer graduates get bothered, but every student it cuts is also a leaver the team never reaches, so the 80 missed today becomes 120.



This is the trade, and it does not go away.

Finding more leavers always means bothering more graduates, and bothering fewer always means finding fewer, so where the school sits is a judgement about students rather than a calculation.

That is why the list goes to a person, and the person decides who to call.

The tool is worse at its job for women

The tool was checked across gender, age, scholarship, and debt, and gender is the one that fails on fairness, because the other three flag groups that really do drop out more.

Share of the students who go on to leave that the tool actually finds.

3 in 10

of the women who go on to drop out are never on the list at all, and for men it is closer to one in eight.

This shows up in no count of who gets flagged.

The tool flags men about twice as often as women, so a check on flagging rates says men are over-contacted and stops there, which answers a different question from the one that matters.

The harm only appears when you count who gets missed rather than who gets contacted.

The students this tool exists to reach are being missed, and they are being missed unevenly.

Closing that gap costs about 30 students

There is a way to correct it. It works. It is not free, and the school should see the bill before it decides.

Leavers found, per 1,000 students.

Thirty fewer students are reached in every thousand.

That is about 8,000 EUR per 1,000 students at a 150 EUR contact, and the catch rate falls from about 8 in 10 to 7 in 10.

What it buys is that those thirty are not loaded onto women.

It does not make the tool better but it makes it evenly bad, and the list of about 410 still fits what the team can reach.

The recommendation is to pay it, but the number is on the table rather than hidden.

The option the school should not take

A second fix closes the same gap and looks better on paper, but it gets there by contacting fewer students of every kind, so it costs seventy leavers instead of thirty.

Fixing a gap by helping fewer people is not a fix.

This is the school's decision, not the tool's.

Thirty students is the price of the tool finding women as well as it finds men, and only the school can say whether that is worth paying.

How to put it in, without betting the school on it

Nothing here asks anyone to trust the tool on day one. Each phase earns the next.

ONE TERM

1. Watch it, do not act on it

The tool produces its list and nobody is contacted, and at the end of term the school compares who it flagged against who actually left, and against who the tutors would have worried about anyway.

If it does not beat the tutors then it does not deploy.

ONE YEAR

2. One department, real contact

A single department runs it for real, and the list goes to a tutor who decides who to call.

This finally tests the thing slide 5 rests on, which is whether reaching out actually keeps anyone, since that was only ever an assumption of three in ten.

ONGOING

3. Full rollout, with a standing check

The tool runs school-wide, with the gender gap re-measured at every intake and reported to whoever owns the decision.

It was measured on a single group of 130 women, and it will not stay fixed on its own.

The tool is cheap to run and expensive to trust, so the school buys that trust in stages.

Three conditions, and the ask

1. **A student never sees their score.**

It appears on no portal and in no letter, and no tutor reads it out, because a prediction handed to a person is not help.

2. **Fix what the school can actually change.**

The strongest signals, a student's course and a parent's job, are not things they can change, so the one that can be is unpaid fees, and a payment plan is cheaper than a tutor and treats the cause.

3. **Re-measure the gap every intake.**

It was measured only once, on a single group, so it belongs in the annual reporting line with a named owner.

THE ASK

Approve phase one.

For one term the tool runs and nobody is contacted, so that by the end we know whether it beats a tutor's judgement.

It costs a term of nobody's time, and it buys the one thing worth buying at this stage, which is evidence.

And approve the shape of the thing.

It should be a ranked list that a person reads and acts on, rather than a label stuck on a student's record that follows them around.

The tool works, and it should not be trusted blindly, but it does not need to be, because it only has to point a tutor at the right student early enough to matter.

The explanation layer, and the check that guards it

Step 9, the generative-AI bonus. Turning a risk score no tutor can read into a sentence they can act on, safely.

Jeryl Donato Estopace

PGD AI/ML capstone, Pillar 5

Predicting student dropout, then auditing the model for bias before anyone trusts it

Who this is for, and what it does

WHO THIS IS FOR

The note is written for a **university tutoring team**, the people who phone a student the model has flagged. The method is written for anyone grading this project, because every claim on the next slides can be run and checked in the notebook.

WHAT IT DOES

Step 5 leaves the model explaining itself in **SHAP values**, and a tutor cannot act on a SHAP value. This layer turns a student's top drivers into two plain sentences they can act on, without leaking anything the fairness audit forbids.

```
one flagged student, 256:

Tuition fees up to date   +2.50
Course                    +1.98
Debtor                    +0.77
mature entry              +0.68
Gender                    +0.43
```

The model's own explanation for one student, from Step 5. This is the input. It has to become two sentences a tutor can read, and nothing more.

DATA AND SCOPE

The data comes from Instituto Politecnico de Portalegre, Portugal, published in 2021, with student outcomes running through about 2019. The model is fitted to that one institution, so every number and fairness finding here holds for an IPP-style Portuguese polytechnic and does not transfer unchanged to a different school.

The method, two writers and a check

One worked example throughout, student 256. Every snippet is copied from an executed cell.

Two ways to write the sentence.

A rules table. A lookup from a driver to a fixed clause, deterministic, and structurally unable to invent anything.

A language model. Handed the student's drivers and asked for two sentences, fluent, and able to say anything at all.

Two prompts for the model. A naive one with no rules, and a guarded one with the four rules on the right.

A verifier. A prompt only asks. So a script then checks each note actually obeyed all four.

`GUARDED_SYSTEM, the four rules`

```
1. Write NO numbers of any kind.
2. Mention ONLY the drivers given.
3. Use no scoring vocabulary.
4. NEVER mention gender.
```

The rules table gets all four for free. The language model has to be asked, and asking is not the same as enforcing.

Finding one, with no rules it does harm

The naive prompt on student 256, straight from the committed cache.

```
student 256, naive note:

Gender: The student's gender (represented
  by "1") statistically correlates with a
  higher dropout rate within this dataset.

... flagged as high-risk ... Course 9119 ...
```

It names the student's gender, prints raw values and a course code, and labels them high-risk. Every one is a harm Step 5 spent a whole notebook forbidding.

The verifier flags four rules at once on this one note. A gender word, a digit, a scoring word, and a driver the model was never handed.

0 / 5

naive notes safe to read to a student

Finding two, four rules make it safe

The same student, the same data, the guarded prompt.

```
student 256, guarded note:

It looks like this student is facing some
financial pressure with outstanding tuition
fees and debt, which can make staying on
their specific course much harder. As they
also entered university as a mature student,
it would be wonderful to reach out and see
how we can support them.
```

No number, no course code, no scoring word, and no mention of gender, even though gender is one of this student's drivers. A warm line a tutor can act on.

5 / 5

guarded notes safe to read to a student

Finding three, the check holds on all five

One clean note proves nothing, so a script re-checks every flagged student.

```
def verify(note, allowed):

  DIGIT.search(note)         # rule 1
  SCORE_WORDS.search(note)   # rule 3
  GENDER.search(note)        # rule 4
  driver not in allowed      # rule 2
```

A mechanical check, one line per rule. No model and no judgement, so it cannot be argued with.

```
rules engine    100%
naive prompt      0%
guarded prompt  100%
```

Naive fails all five, guarded passes all five. It replays off a committed cache, so re-running the notebook needs no key. Generating the cache the first time did.

Finding four, necessary but not sufficient

The check catches what it can see. There is one kind of harm it cannot.

a guarded note that still passed:

```
... whose mother's occupation background
    might offer less familiarity with
    higher education ...
```

Three of the five guarded notes turned a parent's occupation code into a claim about the family's schooling. No number and no gender, so the verifier waved it through.

The verifier is necessary. Without it, zero of five notes were safe.

It is not sufficient. A softer, semantic harm slips past a check built to find digits and banned words. That gap needs a human read, not another regex.

Conclusion, the context is the work

What the language model bought, what it cost, and what a school would ship.

It bought fluency, and words for the course code and the parent's occupation the rules table refuses to touch.

It cost one residual harm, the occupation inference on three of five notes, which the verifier cannot catch. So the model never ships alone. The deterministic rules table is what a school would deploy, with the model kept behind the check.

Everything the model knew, it knew because the project told it, in a context block that took the rest of the work to earn.

The context is the work. The generation is the easy part. Generative AI did not replace the fairness audit. It sits on top of it, and it is only trustworthy because that came first.